

UNIVERSIDADE AUTÓNOMA DE LISBOA
LUÍS DE CAMÕES

DEPARTAMENTO DE ENGENHERIAS E DE CIÊNCIAS DA
COMPUTAÇÃO

(Simulador de Navegação Multi-Agente em Ambientes Logísticos)

Relatório Final de Projeto
apresentado no âmbito da Unidade Curricular de Projeto
da Licenciatura em Engenharia Informática

Autores: Guilherme Freire, Sebastián Gimenez, Tiago
Matias Orientador: Professor Doutor Adrian Dediu
Número dos candidatos: 30013047, 30010652, 30013318

Junho de 2026
Lisboa

(Página em Branco)

Resumo

A navegação cooperativa de múltiplos robôs constitui um dos principais desafios na área dos sistemas multi-agente, exigindo mecanismos eficientes de planeamento de trajetórias e resolução de conflitos para garantir uma deslocação segura e eficiente em ambientes partilhados. Neste contexto, o problema Multi-Agent Path Finding (MAPF) assume um papel fundamental, procurando determinar trajetórias livres de colisões para um conjunto de agentes que partilham o mesmo espaço de navegação.

O presente projeto teve como objetivo desenvolver um simulador de navegação multi-robô baseado numa arquitetura modular, integrando algoritmos clássicos de planeamento de trajetórias com técnicas de aprendizagem por reforço. O sistema foi implementado em **Python**, recorrendo ao algoritmo **A*** para o cálculo das trajetórias individuais, a um mecanismo de coordenação baseado em **Prioritized A*** e a um agente de **Q-Learning** para apoiar a resolução local de conflitos entre robôs. A arquitetura desenvolvida encontra-se organizada em módulos independentes responsáveis pela gestão do ambiente, planeamento dos agentes, resolução de conflitos, recolha de métricas e visualização da simulação.

A avaliação experimental foi realizada através de campanhas compostas por diferentes quantidades de robôs e múltiplas execuções utilizando a mesma sequência de *seeds*, permitindo comparar o desempenho das três abordagens implementadas. Os resultados obtidos demonstraram que o algoritmo **A*** apresenta elevada eficiência no planeamento individual, mas revela limitações ao nível da segurança em cenários densos. Por sua vez, o método **Prioritized A*** reduz significativamente o número de colisões, embora à custa de um aumento do *makespan*. A integração do **Q-Learning** permitiu melhorar a resolução local de conflitos, alcançando um compromisso entre eficiência e segurança sem substituir os mecanismos determinísticos de planeamento.

Em conclusão, o sistema desenvolvido demonstrou a viabilidade da integração entre algoritmos clássicos de navegação e aprendizagem por reforço, constituindo uma plataforma modular e extensível para o estudo e avaliação de estratégias de navegação cooperativa em sistemas multi-robô.

Palavras-chave: Multi-Agent Path Finding, navegação multi-robô, **A***, **Q-Learning**, aprendizagem por reforço, resolução de conflitos.

Abstract

Cooperative multi-robot navigation is one of the main challenges in the field of multi-agent systems, requiring efficient path planning and conflict resolution mechanisms to ensure safe and effective movement in shared environments. In this context, the **Multi-Agent Path Finding (MAPF)** problem plays a fundamental role by determining collision-free paths for multiple agents operating within the same environment.

The objective of this project was to develop a modular multi-robot navigation simulator integrating classical path planning algorithms with reinforcement learning techniques. The system was implemented in **Python**, using the **A*** algorithm for individual path planning, a **Prioritized A*** coordination mechanism, and a **Q-Learning** agent to support local conflict resolution between robots. The proposed architecture is organized into independent modules responsible for environment management, agent planning, conflict resolution, metrics collection, and graphical visualization.

The experimental evaluation was carried out through multiple simulation campaigns using different numbers of robots and repeated executions with the same sequence of random seeds, enabling a fair comparison between the three implemented approaches. The results showed that the **A*** algorithm provides highly efficient individual path planning but suffers from safety limitations in dense environments. The **Prioritized A*** approach significantly reduces the number of collisions, although at the cost of an increased makespan. The integration of **Q-Learning** improved local conflict resolution, achieving a balanced trade-off between efficiency and safety without replacing the deterministic planning mechanisms.

In conclusion, the developed system demonstrates the feasibility of combining classical path planning algorithms with reinforcement learning techniques, providing a modular and extensible platform for the study and evaluation of cooperative navigation strategies in multi-robot systems.

Keywords: Multi-Agent Path Finding, Multi-Robot Navigation, A*, Q-Learning, Reinforcement Learning, Conflict Resolution.

Índice

Resumo	3
Abstract	4
Índice	5
Lista de Quadros/Gráficos.....	8
Lista de Fotografias/Ilustrações	8
1 Introdução.....	8
1.1 Contextualização	8
1.2 Objetivos	9
1.3 Estrutura do Relatório	9
2 Estado da Arte	9
2.1 Sistemas Multi-Agente	9
2.2 Armazéns Inteligentes	11
2.3 Planeamento de Trajetórias	13
2.4 Multi-agent Path Finding (MAPF).....	14
2.5 Algoritmo A*	15
2.6 Prioritized A*	18
2.7 Q-Learning	19
2.8 Simulação e Avaliação de Sistemas Multi-Robô	20
3 Analise do Problema	21
3.1 Descrição do Problema	21
3.2 Objetivos do Sistema.....	22
3.3 Requisitos do Sistema	23
3.3.1 Requisitos Funcionais	23
3.3.2 Requisitos não funcionais	24

3.4	Cenário de Aplicação	25
3.5	Tecnologias Utilizadas	26
4	Arquitetura do Sistema.....	27
4.1	Visão Geral da Arquitetura	27
4.2	Descrição dos Módulos	28
4.2.1	Módulo Config.....	29
4.2.2	Módulo Enviroment	30
4.2.3	Módulo Agents.....	31
4.2.4	Módulo Conflict Resolution.....	33
4.2.5	Módulo Metrics	34
4.2.6	Modulo Viewer	35
5	Implementação	36
5.1	Introdução à Implementação	37
5.2	Implementação do Ambiente de Simulação.....	37
5.2.1	Representação da Grelha.....	38
5.2.2	Carregamento dos Mapas.....	40
5.2.3	Motor da Simulação	41
5.3	Implementação dos Agentes	43
5.3.1	Classe Robot.....	43
5.3.2	Implementação do Algoritmo A*	44
5.3.3	Planeamento Multi-Robô.....	46
5.4	Implementação da Resolução de Conflitos	47
5.4.1	Contexto de Conflito	47
5.4.2	Ações e Sistema de Recompensas.....	48
5.4.3	Implementação do Q-Learning	49
5.5	Implementação do Sistema de Métricas.....	51

5.6	Implementação da Interface Gráfica	53
6	Avaliação Experimental.....	54
6.1	Metodologia Experimental.....	54
6.2	Configuração das Experiências	55
6.3	Comparação entre os Algoritmos.....	56
6.3.1	Taxa de Conclusão	56
6.3.2	Taxa de Sucesso Seguro	57
6.3.3	Número Médio de Colisões	57
6.3.4	Makespan Médio	58
6.4	Discussão dos Resultados	58
6.5	Limitações da Avaliação Experimental	59
7	Conclusões e Trabalho Futuro	60
7.1	Conclusões	60
7.2	Trabalho Futuro.....	61
8	Bibliography	61

Lista de Quadros/Gráficos

Tabela 2.1 – Comparação entre um armazém tradicional e um armazém inteligente	12
Tabela 2.2 – Comparação entre o algoritmo A* e o Prioritized A*	18
Tabela 3.1 - Objetivos específicos do sistema.	23
Tabela 3.2 - Requisitos funcionais.	24
Tabela 3.3 – Requisitos não funcionais.....	24
Tabela 3.4 – Tecnologias utilizadas no desenvolvimento do Sistema.....	26
Tabela 4.1 – Principais módulos do Sistema.....	29
Tabela 4.2 - Principais componentes do módulo Agents.	32
Tabela 4.3 - Principais responsabilidades do módulo Metrics.	34
Tabela 6.1 - Configuração da campanha experimental.	55
Tabela 6.2 - Taxa média de conclusão para os diferentes algoritmos.	56
Tabela 6.3 - Taxa média de sucesso seguro.	57
Tabela 6.4 - Número médio de colisões por execução.	57
Tabela 6.5 - Makespan médio obtido para cada algoritmo	58

Lista de Fotografias/Ilustrações

Figura 2.1 - Exemplo conceptual de um Sistema Multi-Agente	10
Figura 2.2– Exemplo de um armazém inteligente com robôs móveis autônomos	11
Figura 2.3 - Exemplo de uma representação em grelha utilizada no planeamento de trajetórias.	13
Figura 2.4– Exemplo de um conflito de navegação entre dois agentes.....	15
Figura 2.5 – Fluxograma simplificado do algoritmo A*.....	16
Figura 2.6 – Ciclo de interação entre o agente e o ambiente no algoritmo Q-Learning.....	20
Figura 3.1– Ambiente de simulação desenvolvido para a navegação de múltiplos robôs em armazém inteligente	22
Figura 3.2 – – Cenário de aplicação utilizado para a simulação de um armazém inteligente com múltiplos robôs autônomos. Fonte: Elaboração própria.....	25
Figura 4.1 - Arquitetura geral do sistema desenvolvido.	28
Figura 4.2 – – Organização do módulo Environment.	30
Figura 4.4 - Organização do módulo Conflict Resolution.	33

1 Introdução

1.1 Contextualização

A navegação cooperativa de múltiplos robôs é uma área de investigação relevante nos sistemas multi-agente, com aplicações em logística, robótica industrial, veículos autônomos e armazéns inteligentes. Nestes cenários, vários robôs partilham o mesmo ambiente, sendo necessário planear trajetórias eficientes e evitar colisões durante a execução das suas tarefas. O problema **Multi-Agent Path Finding (MAPF)** aborda precisamente este desafio, procurando coordenar múltiplos agentes de forma segura e eficiente.

1.2 Objetivos

O principal objetivo deste projeto consistiu no desenvolvimento de um simulador de navegação multi-robô capaz de planejar trajetórias, coordenar agentes e resolver conflitos de navegação. Para tal, foi implementada uma arquitetura modular em Python, integrando o algoritmo **A*** para planejamento de trajetórias, uma abordagem baseada em **Prioritized A*** para coordenação entre robôs e um agente de **Q-Learning** para apoio à resolução local de conflitos. Adicionalmente, foi desenvolvido um sistema de métricas para avaliar o desempenho das diferentes abordagens e uma interface gráfica para visualização da simulação

1.3 Estrutura do Relatório

O presente relatório encontra-se organizado em sete capítulos. O Capítulo 2 apresenta a fundamentação teórica necessária ao desenvolvimento do projeto. O Capítulo 3 descreve a análise e concepção do sistema. O Capítulo 4 apresenta a arquitetura e organização dos módulos desenvolvidos. O Capítulo 5 descreve a implementação dos principais componentes da aplicação. O Capítulo 6 apresenta a avaliação experimental e a análise dos resultados obtidos. Por fim, o Capítulo 7 reúne as conclusões do trabalho realizado e apresenta possíveis desenvolvimentos futuros.

2 Estado da Arte

2.1 Sistemas Multi-Agente

Os Sistemas Multi-Agente (*Multi-Agent Systems – MAS*) são sistemas compostos por múltiplos agentes autônomos que interagem entre si e com o ambiente para alcançar um ou mais objetivos comuns. Um agente pode ser definido como uma entidade computacional capaz de perceber o ambiente, tomar decisões e executar ações de forma autônoma, contribuindo para o funcionamento global do sistema (Wooldridge, 2009) [1].

Ao contrário dos sistemas baseados num único agente, os Sistemas Multi-Agente distribuem as tarefas por diferentes entidades, permitindo que diversas operações sejam executadas em simultâneo. Esta abordagem melhora a escalabilidade do sistema, aumenta a flexibilidade na execução das tarefas e permite uma melhor adaptação a ambientes dinâmicos, onde diferentes agentes necessitam de cooperar ou competir para atingir os seus objetivos (Jennings, 2001) [2].

As principais vantagens dos Sistemas Multi-Agente incluem:

- Distribuição de tarefas entre múltiplos agentes;
- Execução paralela de operações;
- Maior escalabilidade;
- Melhor utilização dos recursos disponíveis;
- Maior tolerância a falhas individuais.

Atualmente, os Sistemas Multi-Agente encontram aplicação em diversas áreas da Engenharia e da Ciência da Computação, incluindo robótica colaborativa, logística inteligente, sistemas de transporte autônomo, redes de sensores e automação industrial. Em todos estes domínios, a coordenação entre agentes é fundamental para garantir um funcionamento eficiente, seguro e robusto.

A **Figura 2.1** apresenta um exemplo conceptual de um Sistema Multi-Agente, onde vários robôs autónomos partilham o mesmo ambiente e executam tarefas em simultâneo. Embora cada agente possua capacidade de decisão própria, é necessária coordenação para evitar conflitos e assegurar que os objetivos individuais contribuem para o desempenho global do sistema

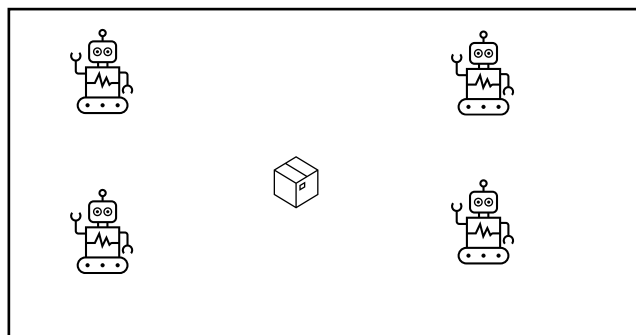


Figura 2.1 - Exemplo conceptual de um Sistema Multi-Agente

Fonte: Elaboração própria.

O projeto desenvolvido neste trabalho baseia-se neste paradigma. Cada robô representa um agente autónomo responsável por transportar mercadorias entre diferentes posições do armazém. Como todos os agentes partilham o mesmo ambiente, torna-se necessário coordenar os seus movimentos, evitar colisões e garantir uma utilização eficiente dos recursos disponíveis. Estes desafios motivam a utilização de algoritmos específicos de planeamento e coordenação, que serão abordados nas secções seguintes.

2.2 Armazéns Inteligentes

O desenvolvimento da Indústria 4.0 e o crescimento do comércio eletrônico impulsionaram a evolução dos sistemas logísticos, conduzindo ao aparecimento dos Armazéns Inteligentes (*Smart Warehouses*) [3]. Estes ambientes integram robôs móveis, sensores, sistemas de gestão de armazéns (*Warehouse Management Systems – WMS*) e algoritmos de otimização, permitindo automatizar operações de armazenamento, transporte e distribuição de mercadorias.

Nos armazéns tradicionais, muitas tarefas dependem da intervenção humana, incluindo a recolha, transporte e organização de produtos. Em contraste, os armazéns inteligentes recorrem a sistemas automatizados capazes de executar estas operações de forma autónoma ou semiautónoma, aumentando a eficiência, reduzindo erros operacionais e melhorando a produtividade global.

A **Figura 2.2** apresenta um exemplo de um armazém inteligente equipado com robôs móveis autónomos. Nestes ambientes, centenas de robôs podem operar simultaneamente, transportando mercadorias entre diferentes zonas do armazém e colaborando para otimizar o fluxo logístico.

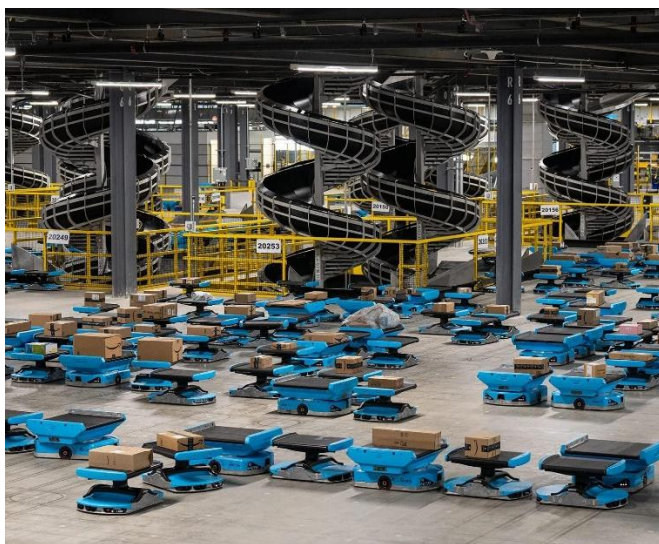


Figura 2.2– Exemplo de um armazém inteligente com robôs móveis autónomos

Fonte: Amazon. Amazon Jobs – Future of Robotics. Disponível em: <https://amazon.jobs/content/en/teams/ftw>.

Os Robôs Móveis Autónomos (*Autonomous Mobile Robots – AMR*) desempenham um papel central nestes sistemas. Para executarem as suas tarefas de forma eficiente, necessitam de

calcular trajetórias seguras, evitar obstáculos e coordenar os seus movimentos com os restantes robôs que operam no mesmo ambiente. À medida que aumenta o número de robôs, cresce também a complexidade da coordenação e da gestão do tráfego dentro do armazém. [4]

O **Quadro 2.1** apresenta uma comparação entre um armazém tradicional e um armazém inteligente, evidenciando algumas das principais diferenças entre ambos os modelos.

Tabela 2.1– Comparação entre um armazém tradicional e um armazém inteligente

Fonte:Elaboração propia.

Característica	Armazém Tradicional	Armazém Inteligente
Transporte de mercadorias	Manual	Robôs móveis autónomos (AMR)
Gestão de inventário	Manual ou parcialmente automatizada	Automatizada
Navegação	Operadores humanos	Algoritmos de planeamento
Produtividade	Moderada	Elevada
Escalabilidade	Limitada	Elevada
Probabilidade de erro	Superior	Inferior

A automatização dos processos logísticos permite reduzir os tempos de operação, otimizar a utilização dos recursos e aumentar a produtividade. No entanto, estes benefícios dependem da existência de algoritmos capazes de coordenar eficientemente múltiplos robôs e resolver conflitos durante a navegação.

O simulador desenvolvido neste projeto inspira-se neste tipo de ambiente. O sistema representa um armazém através de uma grelha bidimensional, onde vários robôs executam tarefas de transporte entre posições de origem e destino. Embora constitua uma representação simplificada de um sistema logístico real, o simulador reproduz desafios fundamentais dos armazéns inteligentes, como o planeamento de trajetórias, a coordenação entre agentes e a prevenção de conflitos.

2.3 Planeamento de Trajetórias

O Planeamento de Trajetórias (*Path Planning*) consiste no processo de determinar um percurso entre uma posição inicial e uma posição de destino, garantindo que o agente consegue atingir o objetivo de forma segura e eficiente. Este problema constitui uma das áreas fundamentais da Robótica e da Inteligência Artificial, estando presente em aplicações como robôs móveis, veículos autónomos, drones e sistemas logísticos automatizados [5].

Para calcular uma trajetória, é necessário dispor de uma representação do ambiente onde o agente se desloca. Uma das abordagens mais utilizadas consiste na representação do espaço através de uma **grelha bidimensional** (*grid*), na qual cada célula representa uma posição discreta do ambiente. Dependendo das características do mapa, cada célula pode representar uma área livre ou um obstáculo, permitindo modelar o espaço de forma simples e eficiente [6].

A **Figura 2.3** apresenta um exemplo de uma representação em grelha utilizada no planeamento de trajetórias. Neste exemplo, o robô deve encontrar um percurso entre a posição inicial (*Start*) e a posição de destino (*Goal*), evitando as células ocupadas por obstáculos.

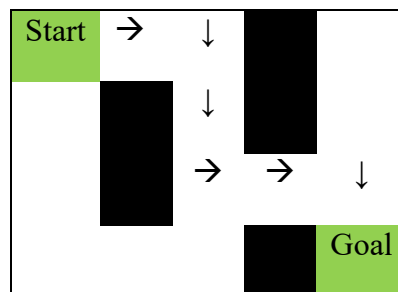


Figura 2.3 - Exemplo de uma representação em grelha utilizada no planeamento de trajetórias.

Fonte: Elaboração própria.

A partir desta representação, um algoritmo de planeamento procura determinar a sequência de movimentos necessária para conduzir o agente até ao destino. Dependendo da aplicação, o percurso obtido pode privilegiar diferentes critérios, como a menor distância percorrida, o menor tempo de execução, o menor consumo energético ou a redução do número de movimentos necessários.

Entre os diversos algoritmos desenvolvidos para resolver este problema, o algoritmo **A*** destaca-se pela sua capacidade de combinar o custo já percorrido com uma estimativa heurística

do custo restante até ao objetivo, permitindo encontrar trajetórias eficientes em ambientes representados por grelhas [7]. O funcionamento deste algoritmo será descrito com maior detalhe na Secção 2.5.

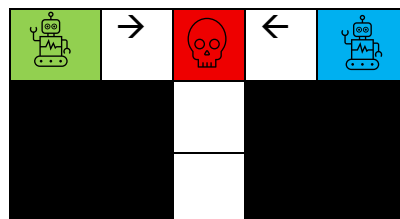
No contexto deste projeto, o ambiente é representado através de mapas no formato MovingAI, convertidos para uma grelha bidimensional onde os robôs calculam os seus percursos entre diferentes posições do armazém. Esta representação constitui a base para a implementação dos algoritmos de navegação utilizados no simulador e prepara a introdução do problema de Multi-Agent Path Finding (**MAPF**), apresentado na secção seguinte.

2.4 Multi-agent Path Finding (MAPF)

O Multi-Agent Path Finding (MAPF) corresponde ao problema de determinar trajetórias para múltiplos agentes que partilham o mesmo ambiente, garantindo que todos conseguem atingir os respetivos destinos sem provocar conflitos durante a navegação. Este problema constitui uma extensão do planeamento de trajetórias para um único agente, introduzindo novos desafios relacionados com a coordenação e sincronização dos movimentos dos diferentes agentes [8].

Ao contrário do planeamento de trajetórias tradicional, onde apenas é necessário encontrar um percurso entre a posição inicial e o destino, no MAPF cada agente deve considerar também a presença e o comportamento dos restantes agentes. Assim, um percurso considerado ótimo para um robô pode tornar-se inviável caso outro robô ocupe a mesma posição ou pretenda utilizar o mesmo caminho no mesmo instante temporal [9].

A **Figura 2.4** apresenta um exemplo de um conflito entre dois robôs num problema de Multi-Agent Path Finding. Neste cenário, ambos os agentes pretendem atravessar a mesma posição da grelha, originando um conflito que deverá ser resolvido através de mecanismos de coordenação.



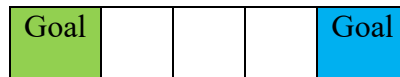


Figura 2.4– Exemplo de um conflito de navegação entre dois agentes

Fonte: Elaboração própria.

Os conflitos mais comuns em problemas MAPF incluem:

- **Conflito de vértice**, quando dois agentes tentam ocupar a mesma posição no mesmo instante.
- **Conflito de aresta**, quando dois agentes trocam simultaneamente de posição utilizando a mesma ligação em sentidos opostos.
- **Conflitos de seguimento, de seguimento**, quando um agente tenta ocupar imediatamente a posição anteriormente ocupada por outro agente, dependendo das restrições impostas pelo sistema.

A resolução destes conflitos pode recorrer a diferentes estratégias, como o replaneamento de trajetórias, a definição de prioridades entre agentes ou a utilização de algoritmos específicos para coordenação multi-agente. A escolha da estratégia influencia diretamente o desempenho do sistema, nomeadamente o tempo necessário para concluir as tarefas, o número de conflitos e a eficiência global da navegação [10].

No contexto deste projeto, o problema MAPF constitui o principal desafio abordado pelo simulador. Cada robô calcula inicialmente o seu percurso individual, sendo posteriormente necessário coordenar os movimentos de todos os agentes para evitar conflitos e garantir que cada robô consegue atingir o respetivo destino de forma segura e eficiente.

2.5 Algoritmo A*

O algoritmo A* (*A-Star*) é um dos algoritmos de pesquisa heurística mais utilizados na resolução de problemas de planeamento de trajetórias. Proposto por Hart, Nilsson e Raphael

em 1968, combina o custo já percorrido desde a posição inicial com uma estimativa do custo restante até ao destino, permitindo encontrar trajetórias eficientes e, quando utilizada uma heurística admissível, garantir a obtenção de uma solução ótima [7].

O funcionamento do algoritmo baseia-se na exploração progressiva dos nós do espaço de pesquisa. Em cada iteração, o A* seleciona o nó com menor custo estimado e expande os seus vizinhos, atualizando os respetivos custos sempre que é encontrado um percurso mais vantajoso. Este processo repete-se até que o objetivo seja alcançado ou até que deixem de existir nós disponíveis para exploração [5].

O custo total associado a cada nó é determinado pela **Equação (2.1)**.

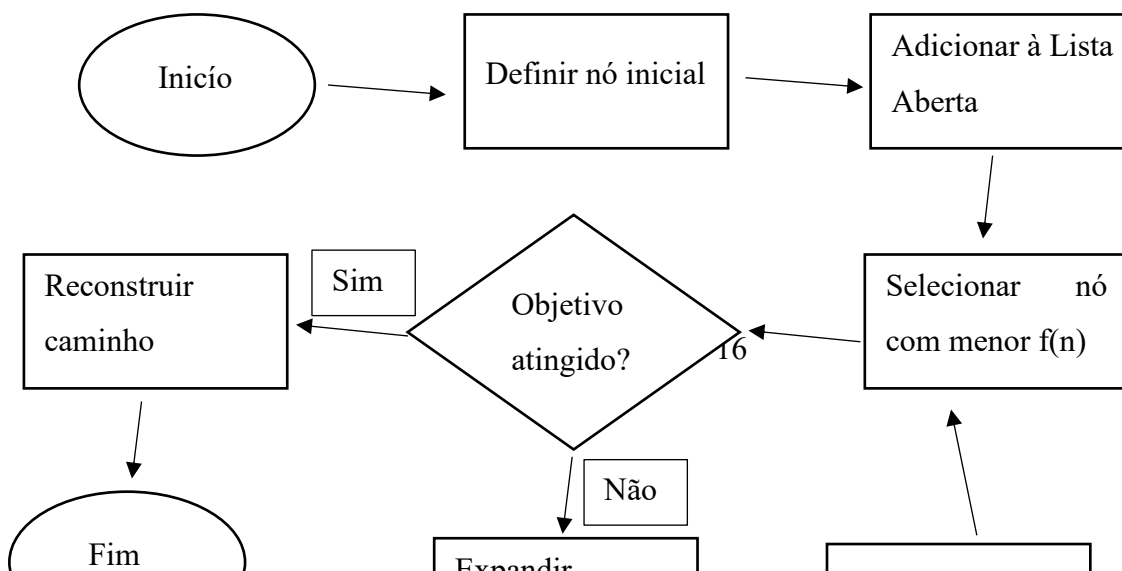
$$f(n) = g(n) + h(n) \quad (2.1)$$

Onde:

- $f(n)$ representa o custo total estimado;
- $g(n)$ corresponde ao custo acumulado desde a posição inicial até ao nó atual;
- $h(n)$ representa a estimativa heurística do custo restante até ao objetivo.

A componente heurística desempenha um papel fundamental no desempenho do algoritmo, uma vez que orienta a pesquisa para as regiões mais promissoras do espaço de procura. Quanto mais precisa for esta estimativa, menor tende a ser o número de nós explorados durante a pesquisa, reduzindo o tempo necessário para encontrar uma solução [5].

A **Figura 2.5** apresenta um fluxograma simplificado do funcionamento do algoritmo A*. O processo inicia-se com a definição do nó inicial, prossegue com a seleção sucessiva do nó com menor valor de $f(n)$ e termina quando o objetivo é alcançado ou quando deixa de existir um percurso válido.



Entre as principais vantagens do algoritmo A* destacam-se:

- Capacidade de encontrar trajetórias ótimas quando utilizada uma heurística admissível;
- Redução do número de nós explorados relativamente a algoritmos de pesquisa não heurísticos;
- Elevada eficiência em ambientes discretos representados por grelhas;
- Facilidade de implementação e adaptação a diferentes aplicações.

Apesar das suas vantagens, o algoritmo A* foi concebido para o planeamento de trajetórias de um único agente. Quando vários agentes partilham o mesmo ambiente, o algoritmo deixa de considerar possíveis conflitos entre robôs, tornando necessária a utilização de estratégias adicionais de coordenação. Esta limitação motiva a utilização de algoritmos específicos para ambientes multi-agente, como o Prioritized A*, apresentado na secção seguinte [10].

Neste trabalho, o algoritmo A* foi implementado recorrendo a uma representação espaço-temporal dos estados, permitindo considerar simultaneamente a posição de cada robô e o instante temporal associado durante o planeamento das trajetórias. A implementação suporta ainda restrições temporais sobre vértices e arestas, bem como a possibilidade de permanência na mesma posição (*wait*), tornando o algoritmo adequado para cenários com múltiplos robôs. Esta abordagem constitui a base do sistema de navegação desenvolvido, sendo posteriormente complementada por mecanismos de coordenação e resolução de conflitos durante a execução da simulação.

2.6 Prioritized A*

O Prioritized A* é uma abordagem utilizada para resolver problemas de Multi-Agent Path Finding (MAPF), combinando o algoritmo A* com um mecanismo de atribuição de prioridades entre agentes. Nesta estratégia, o algoritmo A* é utilizado para calcular as trajetórias individuais, enquanto a coordenação entre múltiplos robôs é assegurada através de um planejamento sequencial baseado em prioridades [7], [8].

Nesta abordagem, os agentes são ordenados segundo uma prioridade previamente definida. O robô com maior prioridade calcula primeiro o seu percurso, sem considerar os restantes agentes. Posteriormente, cada robô de prioridade inferior calcula a sua trajetória utilizando o algoritmo A*, tendo em consideração os caminhos já reservados pelos agentes anteriormente planeados. Desta forma, as posições ocupadas pelos robôs de maior prioridade passam a constituir restrições temporais para os restantes agentes [10].

Esta estratégia permite reduzir significativamente a ocorrência de conflitos durante a navegação, evitando situações em que dois robôs ocupam a mesma posição ou atravessam a mesma aresta em sentidos opostos no mesmo instante temporal. Apesar de não garantir uma solução ótima para todos os agentes, o Prioritized A* apresenta um bom compromisso entre qualidade das soluções e eficiência computacional, sendo frequentemente utilizado em aplicações práticas de navegação multi-robô [10].

O **Quadro 2.2** apresenta uma comparação entre o algoritmo A* tradicional e o Prioritized A*, evidenciando as principais diferenças entre ambas as abordagens.

Tabela 2.2 – Comparação entre o algoritmo A e o Prioritized A*.*

Fonte: Elaboração própria.

Critério	A*	Prioritized A*
Número de agentes	Um agente	Múltiplos agentes
Planeamento	Individual	Sequencial por prioridade
Considera conflitos entre robôs	Não	Sim
Complexidade	Menor	Maior
Otimalidade global	Possível para um agente	Não garantida
Aplicação no projeto	Cálculo de caminhos individuais	Coordenação entre robôs

Apesar da sua simplicidade e facilidade de implementação, o desempenho do Prioritized A* depende da ordem das prioridades atribuídas aos agentes. Uma escolha inadequada pode originar percursos mais longos para alguns robôs ou mesmo impedir que determinados agentes encontrem uma solução viável sem recorrer a mecanismos adicionais de replaneamento [10].

A arquitetura desenvolvida utiliza o Prioritized A* como principal estratégia de coordenação entre os robôs. Cada agente calcula inicialmente o seu percurso através do algoritmo A*, sendo posteriormente aplicadas restrições temporais e mecanismos de coordenação durante a execução da simulação, permitindo reduzir conflitos e melhorar a eficiência da navegação em ambientes com múltiplos robôs.

2.7 Q-Learning

O Q-Learning é um algoritmo de Aprendizagem por Reforço (*Reinforcement Learning*) que permite a um agente aprender uma política de decisão através da interação com o ambiente. Ao contrário dos algoritmos apresentados nas secções anteriores, que calculam explicitamente uma trajetória antes da execução, o Q-Learning aprende progressivamente quais as ações mais vantajosas com base na experiência adquirida durante sucessivas interações [11].

Em cada estado do ambiente, o agente seleciona uma ação, observa o estado seguinte e recebe uma recompensa (*reward*) que avalia a qualidade da decisão tomada. Ao longo do processo de aprendizagem, os valores associados a cada par estado-ação são continuamente atualizados, permitindo que o agente melhore gradualmente o seu comportamento [12].

A atualização dos valores da função Q é efetuada de acordo com a **Equação (2.2)**.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.2)$$

Onde:

- $Q(s, a)$ representa o valor estimado (*Q-value*) associado à execução da ação *a* no estado *s*;
- α corresponde à **taxa de aprendizagem** (*learning rate*), determinando a influência da nova informação na atualização da função Q
- r representa a **recompensa** (*reward*) obtida após a execução da ação;
- γ corresponde ao **fator de desconto** (*discount factor*), responsável por ponderar a importância das recompensas futuras;
- $\max_{a'} Q(s', a')$ representa o maior valor estimado para o estado seguinte s' , considerando todas as ações possíveis;

- s' representa o **estado seguinte** alcançado após a execução da ação
- a' representa uma **ação possível** que pode ser executada no estado seguinte.

A **Figura 2.6** apresenta o ciclo de interação entre o agente e o ambiente durante o processo de aprendizagem por reforço. Em cada iteração, o agente executa uma ação, observa o novo estado e recebe uma recompensa, utilizando essa informação para atualizar o conhecimento adquirido.

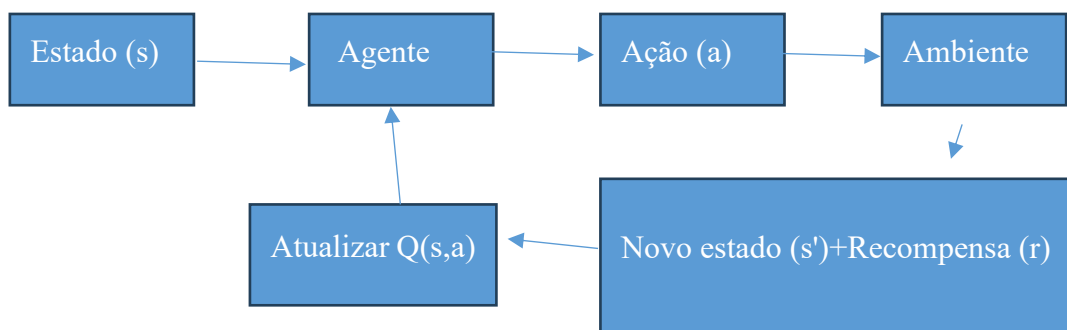


Figura 2.6 – Ciclo de interação entre o agente e o ambiente no algoritmo Q-Learning.

1. Fonte: Elaboração própria.

le aprender

diretamente a partir da experiência, a adaptação a ambientes dinâmicos e a possibilidade de melhorar continuamente o desempenho sem necessidade de um modelo completo do ambiente. No entanto, o processo de aprendizagem pode exigir um elevado número de episódios até atingir um comportamento estável, especialmente em ambientes com um grande número de estados [11].

Para complementar o algoritmo A*, foi integrada uma abordagem baseada em Q-Learning destinada à resolução local de conflitos entre robôs. Em vez de substituir o planejamento das trajetórias, o agente de aprendizagem por reforço seleciona ações como WAIT (esperar) ou YIELD (ceder passagem) durante situações de conflito, aprendendo progressivamente quais as decisões que conduzem a uma navegação mais eficiente e segura.

2.8 Simulação e Avaliação de Sistemas Multi-Robô

A simulação desempenha um papel fundamental no desenvolvimento e avaliação de sistemas multi-robô, permitindo testar algoritmos de navegação, coordenação e resolução de conflitos sem necessidade de recorrer a hardware físico. A utilização de ambientes simulados

possibilita a reprodução de diferentes cenários de teste, reduzindo custos de desenvolvimento e facilitando a análise do comportamento dos robôs em condições controladas [6].

No contexto da robótica móvel, a simulação permite avaliar o desempenho dos algoritmos de planeamento de trajetórias antes da sua implementação em sistemas reais. Através deste processo, é possível medir indicadores relacionados com a eficiência da navegação, o tempo de execução, a ocorrência de conflitos e o comportamento dos robôs perante diferentes configurações do ambiente [6].

Com base nestes princípios, foi desenvolvido um simulador próprio que permite representar ambientes inspirados em armazéns inteligentes e executar múltiplas experiências de forma controlada. O sistema integra mecanismos de recolha e exportação de métricas, possibilitando comparar objetivamente o desempenho das diferentes estratégias de navegação e resolução de conflitos implementadas.

A utilização de um simulador próprio oferece ainda maior flexibilidade na experimentação de novas funcionalidades e algoritmos, permitindo realizar testes repetíveis em diferentes cenários e fornecer uma base consistente para a avaliação experimental apresentada nos capítulos seguintes.

3 Analise do Problema

3.1 Descrição do Problema

A crescente adoção de robôs móveis autónomos em centros logísticos e armazéns inteligentes tem permitido automatizar tarefas de transporte de mercadorias, aumentando a eficiência operacional e reduzindo a necessidade de intervenção humana. No entanto, a coordenação simultânea de vários robôs num mesmo ambiente continua a representar um desafio significativo, uma vez que estes partilham corredores, cruzamentos e zonas de circulação comuns.

O problema abordado neste projeto consiste em permitir que um conjunto de robôs navegue autonomamente num ambiente representado por uma grelha, deslocando-se entre posições iniciais e objetivos previamente definidos. Durante a execução destas tarefas podem ocorrer situações de conflito, nomeadamente quando dois ou mais robôs pretendem ocupar a mesma célula ou atravessar um corredor em sentidos opostos, originando bloqueios, atrasos ou colisões.

Embora algoritmos de planeamento, como o A*, permitam calcular trajetórias eficientes para cada robô individualmente, a sua aplicação isolada não garante uma navegação eficiente

em cenários multi-agente. Torna-se, por isso, necessário complementar o planejamento inicial com mecanismos capazes de coordenar o comportamento dos robôs durante a execução da simulação, reduzindo conflitos e melhorando o desempenho global do sistema.

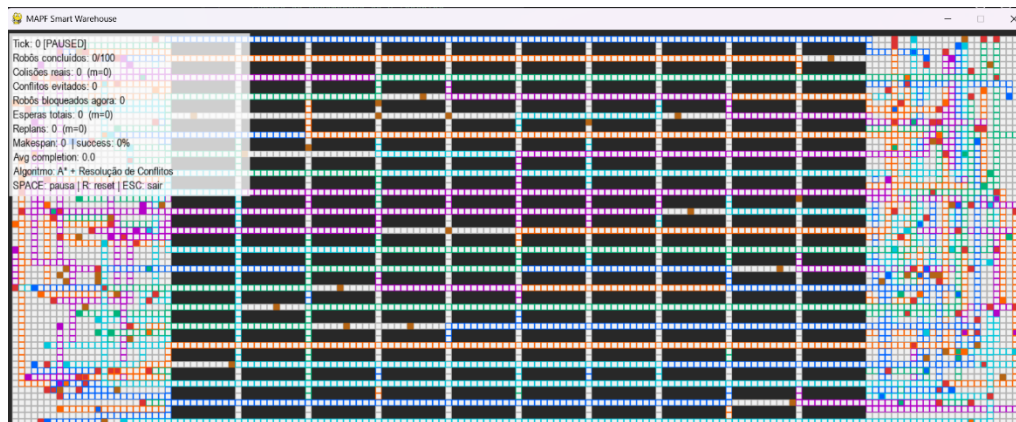


Figura 3.1– Ambiente de simulação desenvolvido para a navegação de múltiplos robôs em armazém inteligente

Fonte: Elaboração própria.

Com o objetivo de estudar este problema, foi desenvolvido um simulador capaz de representar ambientes inspirados em armazéns inteligentes, permitindo a execução simultânea de múltiplos robôs autônomos e a comparação de diferentes estratégias de navegação. O sistema integra algoritmos de planejamento de trajetórias, mecanismos de resolução de conflitos e um módulo de recolha de métricas, possibilitando avaliar indicadores como colisões, replaneamentos, tempos de execução e taxa de sucesso das tarefas.

3.2 Objetivos do Sistema

O sistema desenvolvido tem como principal objetivo disponibilizar um ambiente de simulação que permita estudar e comparar diferentes estratégias de navegação para múltiplos robôs autônomos em cenários inspirados em armazéns inteligentes. Para esse efeito, foram integrados algoritmos de planejamento de trajetórias, mecanismos de resolução de conflitos e ferramentas de recolha de métricas, possibilitando a avaliação do desempenho das diferentes abordagens implementadas.

De forma mais específica, o sistema foi concebido para cumprir os objetivos apresentados no Quadro 3.1.

Tabela 3.1 - Objetivos específicos do sistema.

Objetivo	Descrição
Implementar um ambiente de simulação	Desenvolver um simulador capaz de representar um armazém baseado numa grelha bidimensional.
Planear trajetórias	Implementar algoritmos de planeamento capazes de calcular percursos entre a posição inicial e o objetivo de cada robô.
Coordenar múltiplos robôs	Reduzir conflitos durante a navegação através de estratégias de coordenação entre agentes.
Integrar aprendizagem por reforço	Utilizar Q-Learning como mecanismo complementar para apoiar a resolução local de conflitos.
Recolher métricas	Registar indicadores como colisões, esperas, replaneamentos, makespan e taxa de sucesso.
Comparar algoritmos	Avaliar o desempenho das diferentes abordagens implementadas em vários cenários de teste.

Fonte: Elaboração própria.

O cumprimento destes objetivos permite analisar o comportamento do sistema perante diferentes configurações de ambiente e número de robôs, fornecendo uma base para a comparação entre algoritmos de navegação e para a avaliação da eficácia dos mecanismos de resolução de conflitos implementados

3.3 Requisitos do Sistema

A definição dos requisitos do sistema permitiu estabelecer as funcionalidades essenciais e as características de desempenho necessárias para o desenvolvimento do simulador. Os requisitos foram divididos em duas categorias: requisitos funcionais, que descrevem os serviços disponibilizados pelo sistema, e requisitos não funcionais, que definem propriedades relacionadas com desempenho, usabilidade e manutenção.

3.3.1 Requisitos Funcionais

Os requisitos funcionais correspondem às funcionalidades que o sistema deve disponibilizar ao utilizador e às operações necessárias para cumprir os objetivos definidos.

Tabela 3.2 - Requisitos funcionais.

Fonte: Elaboração própria.

Codigo	Requisito
RF1	Carregar mapas de armazém em formato MovingAI.
RF2	Gerar e gerir múltiplos robôs autónomos no ambiente de simulação.
RF3	Calcular trajetórias utilizando o algoritmo A*.
RF4	Suportar a estratégia Prioritized A* para coordenação entre robôs.
RF5	Integrar um agente baseado em Q-Learning para resolução local de conflitos.
RF6	Detetar conflitos durante a navegação e aplicar mecanismos de resolução.
RF7	Replanear trajetórias quando necessário.
RF8	Recolher métricas de desempenho durante a simulação.
RF9	Exportar os resultados obtidos para ficheiros de análise.
RF10	Disponibilizar uma interface gráfica para visualização da simulação.

3.3.2 Requisitos não funcionais

Os requisitos não funcionais especificam características relacionadas com a qualidade do sistema, influenciando aspetos como desempenho, escalabilidade, manutenção e facilidade de utilização.

Tabela 3.3 – Requisitos não funcionais.

Fonte: Elaboração própria.

Codigo	Requisito
RNF1	O sistema deve apresentar uma arquitetura modular para facilitar a manutenção e a evolução do código.
RNF2	A simulação deve suportar a execução simultânea de múltiplos robôs.
RNF3	Os algoritmos implementados devem poder ser substituídos ou estendidos sem alterar a estrutura principal do sistema.
RNF4	O sistema deve permitir a execução de experiências repetidas para comparação de resultados.
RNF5	As métricas devem ser registadas de forma consistente para posterior análise.
RNF6	A interface gráfica deve permitir acompanhar visualmente o estado da simulação em tempo real.
RNF7	O sistema deve manter uma organização clara dos módulos e ficheiros do projeto.

Os requisitos apresentados serviram como base para o desenvolvimento do simulador e orientaram a implementação das diferentes funcionalidades descritas nos capítulos seguintes. A sua definição permitiu garantir que o sistema responde aos objetivos inicialmente estabelecidos, possibilitando a realização de experiências controladas e a comparação entre diferentes estratégias de navegação.

3.4 Cenário de Aplicação

O cenário considerado neste trabalho representa um armazém inteligente onde múltiplos robôs móveis autónomos executam tarefas de transporte entre diferentes localizações. A área de trabalho é modelada através de uma grelha bidimensional, na qual coexistem zonas navegáveis e obstáculos que definem corredores e condicionam os percursos disponíveis para os robôs.

Cada simulação inicia-se com a atribuição de uma posição inicial e de um destino a cada robô. A partir desse momento, os agentes calculam as respetivas trajetórias e deslocam-se de forma autónoma pelo ambiente. Durante a execução podem surgir situações de conflito resultantes da utilização simultânea de determinadas células ou corredores, tornando necessária a aplicação de mecanismos de coordenação para garantir uma navegação eficiente e segura.

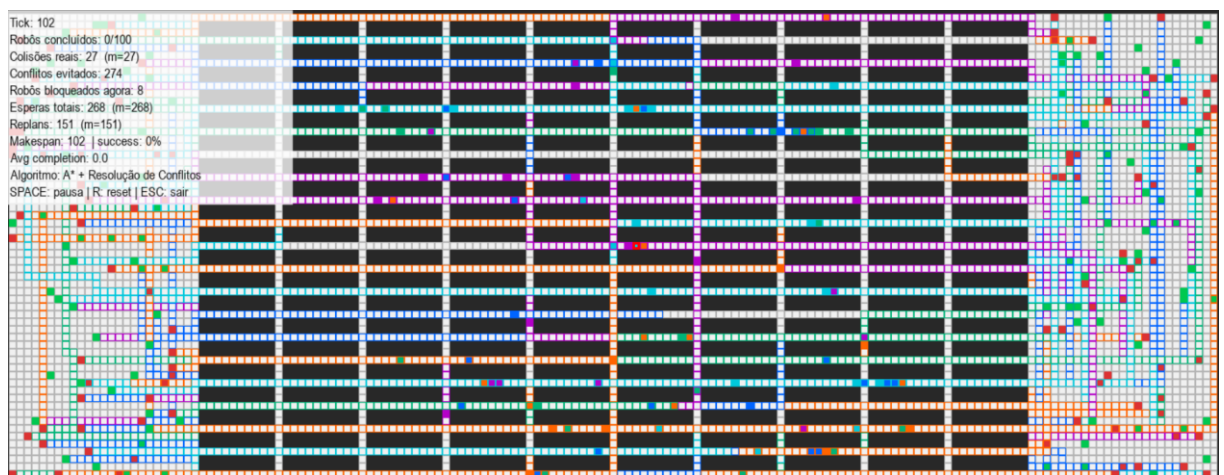


Figura 3.2 - – Cenário de aplicação utilizado para a simulação de um armazém inteligente com múltiplos robôs autónomos.

Fonte: Elaboração própria.

O cenário implementado apresenta as seguintes características:

- Representação do ambiente através de uma grelha bidimensional;
- Utilização de obstáculos para definir corredores e zonas de circulação;
- Navegação simultânea de múltiplos robôs autónomos;

- Planeamento automático de trajetórias entre posições de origem e destino;
- Detecção e resolução de conflitos durante a execução da simulação;
- Recolha automática de métricas para avaliação do desempenho dos algoritmos implementados.

Apesar de representar uma versão simplificada de um armazém inteligente, o ambiente desenvolvido permite reproduzir situações comuns em sistemas logísticos automatizados. Esta abordagem proporciona um contexto controlado para avaliar o comportamento dos algoritmos implementados e analisar o impacto das diferentes estratégias de coordenação na eficiência global da navegação.

3.5 Tecnologias Utilizadas

O desenvolvimento do simulador recorreu a um conjunto de tecnologias que suportam a implementação dos algoritmos de navegação, a representação gráfica do ambiente e a recolha de resultados experimentais. A escolha destas ferramentas teve em consideração a facilidade de desenvolvimento, a disponibilidade de bibliotecas especializadas e a adequação aos objetivos definidos para o projeto.

Tabela 3.4 – Tecnologias utilizadas no desenvolvimento do Sistema.

Fonte: Elaboração própria.

Tecnologia	Finalidade
Python 3	Linguagem de programação utilizada para o desenvolvimento do simulador e implementação dos algoritmos.
Pygame	Biblioteca utilizada para a interface gráfica e visualização da simulação em tempo real.
PyCharm	Ambiente de desenvolvimento integrado (IDE) utilizado na implementação e depuração do projeto.
MovingAI Maps	Formato de mapas utilizado para representar o ambiente do armazém e os obstáculos.
JSON	Armazenamento da Q-Table e de ficheiros de configuração.

CSV	Exportação das métricas e dos resultados das simulações.
Git / GitHub	Controlo de versões e colaboração durante o desenvolvimento do projeto.
Pytest	Desenvolvimento e execução de testes automáticos do sistema.

A utilização destas tecnologias permitiu desenvolver uma solução modular e extensível, facilitando a implementação dos diferentes algoritmos de navegação e a realização de experiências em diversos cenários de teste. A combinação entre Python e Pygame proporcionou um ambiente adequado para a construção do simulador, enquanto a utilização de ficheiros JSON e CSV simplificou o armazenamento e a análise dos resultados obtidos.

4 Arquitetura do Sistema

4.1 Visão Geral da Arquitetura

A arquitetura do sistema foi concebida segundo uma abordagem modular, permitindo separar as diferentes responsabilidades associadas ao funcionamento da simulação. Esta organização facilita a manutenção do código, promove a reutilização dos componentes desenvolvidos e simplifica a integração de novas funcionalidades ao longo da evolução do projeto.

O sistema encontra-se organizado em módulos responsáveis pela configuração da simulação, representação do ambiente, gestão dos robôs, planeamento de trajetórias, resolução de conflitos, recolha de métricas e visualização da execução. Cada módulo desempenha uma função específica, comunicando com os restantes através de interfaces bem definidas, reduzindo o acoplamento entre componentes e tornando a arquitetura mais organizada e extensível. A descrição detalhada de cada módulo é apresentada na Secção 4.2.

Durante a execução da simulação, o sistema inicia o carregamento das configurações e do cenário selecionado, procedendo posteriormente à criação do ambiente e dos robôs. Em seguida, é efetuado o planeamento inicial das trajetórias através dos algoritmos implementados. À medida que a simulação decorre, os robôs deslocam-se autonomamente pelo ambiente, sendo monitorizados os possíveis conflitos entre agentes. Sempre que necessário, são aplicados mecanismos de resolução de conflitos, enquanto o módulo de métricas recolhe informação relativa ao desempenho da simulação para posterior análise.

A Figura 4.1 apresenta uma visão geral da arquitetura do sistema desenvolvido, evidenciando os principais módulos que o constituem e a sua organização funcional.

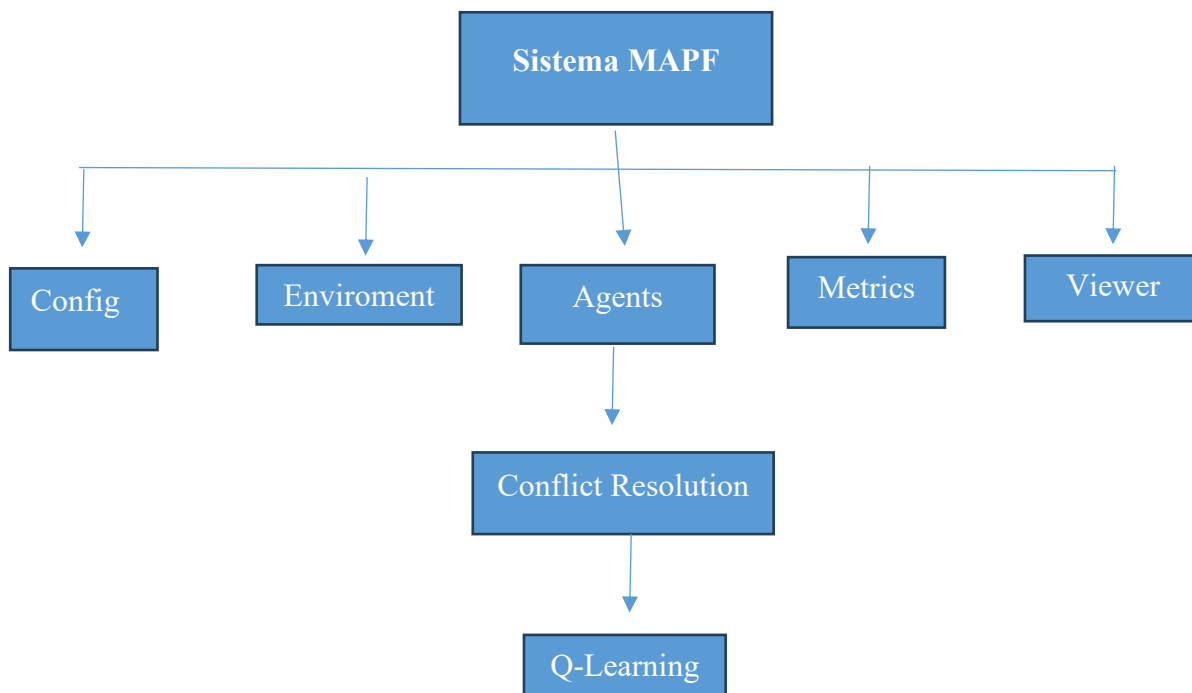


Figura 4.1 - Arquitetura geral do sistema desenvolvido.

Fonte: Elaboração própria.

nte de forma relativamente independente, facilitando a evolução progressiva do sistema e a integração de novas funcionalidades. Esta abordagem contribuiu igualmente para uma estrutura mais organizada, simplificando a manutenção do código e favorecendo futuras extensões da aplicação.

4.2 Descrição dos Módulos

A arquitetura modular adotada permitiu organizar o sistema em diferentes componentes, cada um responsável por uma parte específica do funcionamento da aplicação. Esta divisão facilita a manutenção do código, promove a reutilização de funcionalidades e possibilita a evolução independente de cada módulo, reduzindo o acoplamento entre os diferentes componentes do sistema.

Os módulos encontram-se organizados de acordo com as suas responsabilidades, abrangendo a configuração da simulação, a representação do ambiente, a gestão dos robôs, a resolução de conflitos, a recolha de métricas e a visualização da execução. Esta organização

contribui para uma estrutura de software clara e extensível, simplificando tanto o desenvolvimento como a integração de novas funcionalidades.

O Quadro 4.1 apresenta uma visão geral dos principais módulos que constituem o sistema e a respetiva função.

Tabela 4.1 – Principais módulos do Sistema.

Fonte: Elaboração própria.

Módulo	Função
Config	Armazena os parâmetros de configuração utilizados durante a simulação.
Environment	Representa o ambiente de execução, incluindo a grelha, os obstáculos e o motor da simulação.
Agents	Implementa os robôs e os algoritmos responsáveis pelo planeamento das trajetórias.
Conflict Resolution	Implementa os mecanismos de deteção e resolução de conflitos entre robôs, incluindo estratégias baseadas em Q-Learning.
Metrics	Recolhe, processa e disponibiliza as métricas geradas durante a simulação.
Viewer	Disponibiliza a interface gráfica para visualização da simulação e interação com o utilizador.

4.2.1 Módulo Config

O módulo Config centraliza os parâmetros de configuração utilizados pelos diferentes componentes do sistema. A sua implementação encontra-se no ficheiro settings.py, onde são definidos os valores que controlam a execução da simulação, os algoritmos utilizados, os cenários experimentais e a visualização gráfica.

Entre os principais parâmetros configuráveis encontram-se o algoritmo de navegação ativo, definido através da variável **PLANNER**, o caminho do mapa utilizado pela simulação (**MAP_PATH**), o número de robôs (**NUM_ROBOTS**), a quantidade de obstáculos dinâmicos (**DYNAMIC_OBSTACLES**) e o intervalo temporal entre movimentos (**MOVE_DELAY**).

Estes valores permitem alterar rapidamente o comportamento da simulação sem modificar diretamente os restantes módulos do sistema.

O ficheiro inclui também parâmetros destinados à execução experimental, como **ROBOT_COUNTS**, **MAX_TICKS**, **RUNS_PER_SEED**, **TRAIN_EPISODES** e **TRAIN_ROBOTS**. Estes elementos são utilizados para controlar experiências repetidas, definir diferentes quantidades de robôs e configurar o processo de treino do agente baseado em Q-Learning.

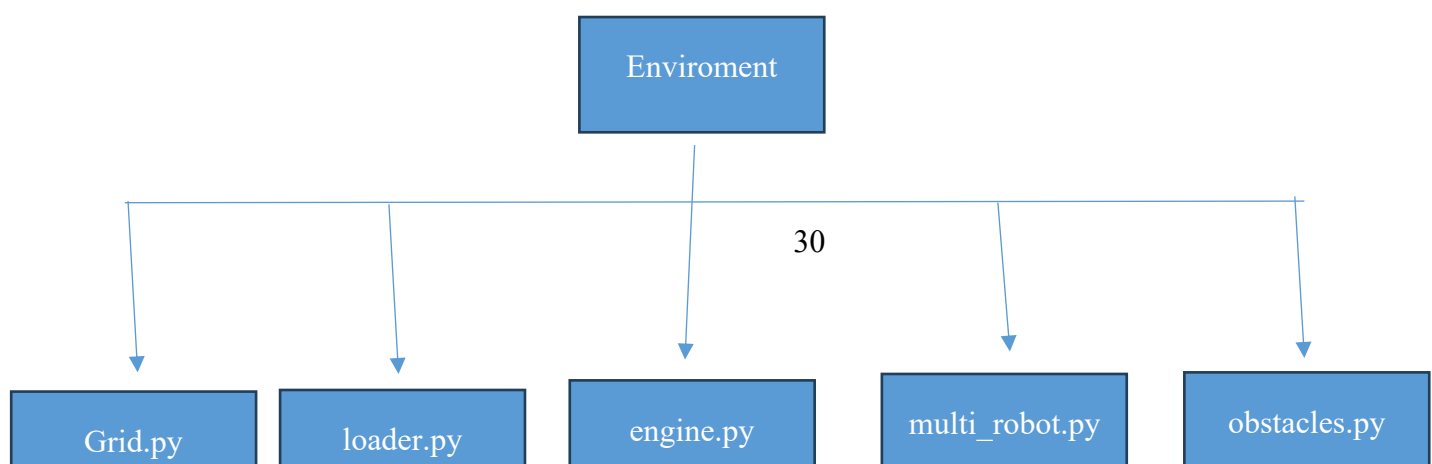
Além das configurações funcionais, o módulo define ainda constantes visuais utilizadas pela interface gráfica, incluindo cores associadas a obstáculos, células livres, grelha, posições iniciais, objetivos, colisões e robôs. Esta centralização permite manter a consistência visual da aplicação e simplifica futuras alterações à apresentação gráfica do simulador.

A utilização de um módulo dedicado à configuração reduz o acoplamento entre componentes, uma vez que os parâmetros principais ficam concentrados num único ficheiro. Desta forma, torna-se possível ajustar cenários, algoritmos e parâmetros experimentais de forma simples e consistente ao longo do desenvolvimento e da avaliação do sistema.

4.2.2 Módulo Enviroment

O módulo Environment é responsável pela representação do ambiente de simulação e pela coordenação da execução global do sistema. Este módulo disponibiliza as estruturas necessárias para modelar o armazém, gerir os robôs, controlar os obstáculos e sincronizar a evolução temporal da simulação, constituindo o núcleo funcional da aplicação.

A implementação encontra-se distribuída pelos ficheiros **grid.py**, **loader.py**, **engine.py**, **multi_robot.py** e **obstacles.py**, cada um responsável por uma componente específica do ambiente. Em conjunto, estes elementos permitem representar a grelha de navegação, carregar mapas no formato **MovingAI**, gerir obstáculos estáticos e dinâmicos, criar e posicionar os robôs através do módulo **multi_robot.py** e coordenar o ciclo de execução da simulação. Esta organização modular facilita a evolução independente de cada componente e reduz o acoplamento entre os diferentes módulos da aplicação.



O ficheiro `engine.py` implementa o motor principal da simulação, sendo responsável por inicializar o ambiente, coordenar o avanço temporal, gerir a execução dos robôs e sincronizar a interação entre os restantes módulos. Durante cada *tick* da simulação, a Engine atualiza o estado global do sistema, processa eventos relevantes e comunica com os módulos Agents, Conflict Resolution, Metrics e Viewer, assegurando o funcionamento consistente de toda a aplicação.

Ao concentrar a gestão do ambiente num módulo dedicado, torna-se possível reutilizar a mesma infraestrutura para diferentes cenários experimentais, variando apenas os mapas utilizados, o número de robôs ou os parâmetros definidos no módulo **Config**. Esta separação entre a representação do ambiente e os restantes componentes contribui para uma arquitetura modular, extensível e de fácil manutenção.

4.2.3 Módulo Agents

O módulo Agents é responsável pela implementação dos robôs e dos algoritmos de planeamento de trajetórias utilizados durante a simulação. Este módulo disponibiliza os componentes necessários para representar cada agente, calcular caminhos entre a posição inicial e o objetivo e executar os movimentos ao longo da navegação.

A implementação encontra-se distribuída pelos ficheiros **robot.py**, **a_star.py**, **prioritized_a_star.py** e **types.py**. O ficheiro **robot.py** implementa a classe **Robot**, responsável por encapsular o estado e o comportamento de cada robô. Para além de armazenar informação

como a posição inicial, posição atual, objetivo, trajetória planeada e estatísticas de execução, esta classe disponibiliza métodos para o planeamento de trajetórias, replaneamento e movimentação ao longo da simulação.

Tabela 4.2 - Principais componentes do módulo Agents.

Fonte: Elaboração própria.

Componente	Responsabilidade
robot.py	Implementação da classe Robot e gestão do estado e comportamento de cada robô.
a_star.py	Implementação do algoritmo A* para o cálculo de trajetórias.
prioritized_a_star.py	Interface compatível para planeamento priorizado baseada no algoritmo A*.
types.py	Definição dos tipos e estruturas de dados utilizados pelo módulo.

O algoritmo de navegação principal encontra-se implementado em **a_star.py**, recorrendo a uma representação espaço-temporal dos estados, na qual cada estado é definido pelo par $((x, y), t)$. Esta abordagem permite considerar simultaneamente a posição do robô e o instante temporal correspondente, suportando restrições temporais sobre vértices e arestas, bem como a ação de permanência na posição atual (*wait*). Estas características tornam o algoritmo adequado para problemas de **Multi-Agent Path Finding (MAPF)** e facilitam a integração de mecanismos de coordenação entre múltiplos robôs.

O ficheiro **prioritized_a_star.py** disponibiliza uma interface compatível com os restantes algoritmos de planeamento, reutilizando a implementação do algoritmo A*. Na arquitetura desenvolvida, a coordenação entre múltiplos robôs, incluindo a deteção de conflitos, esperas, cedências de prioridade e replaneamentos, é efetuada pelo módulo **Environment**, permitindo separar o cálculo inicial das trajetórias da gestão dinâmica da execução da simulação.

A separação entre a representação dos robôs, os algoritmos de planeamento e os mecanismos de coordenação contribui para uma arquitetura modular e extensível. Enquanto o módulo Agents é responsável pelo cálculo das trajetórias individuais, o módulo Environment, através da classe Engine, gere a coordenação entre os robôs durante a execução da simulação,

incluindo a aplicação de restrições temporais, o replaneamento de trajetórias e a detecção de conflitos. Esta separação permite manter distintas as responsabilidades de planeamento e de coordenação, facilitando a evolução independente dos diferentes componentes do sistema.

4.2.4 Módulo Conflict Resolution

O módulo **Conflict Resolution** é responsável pela implementação dos mecanismos de detecção e resolução de conflitos entre robôs durante a execução da simulação. A sua principal função consiste em fornecer uma camada de decisão capaz de responder a conflitos de vértice, trocas frontais (*edge conflicts*) e situações de bloqueio, permitindo que os agentes continuem a sua navegação de forma coordenada.

A implementação encontra-se distribuída pelos ficheiros **actions.py**, **base.py**, **context.py**, **rewards.py** e **q_agent.py**. Esta organização separa as diferentes responsabilidades associadas ao processo de decisão, permitindo representar o estado do conflito, definir as ações disponíveis, calcular recompensas e implementar diferentes **políticas de resolução de conflitos** de forma independente.

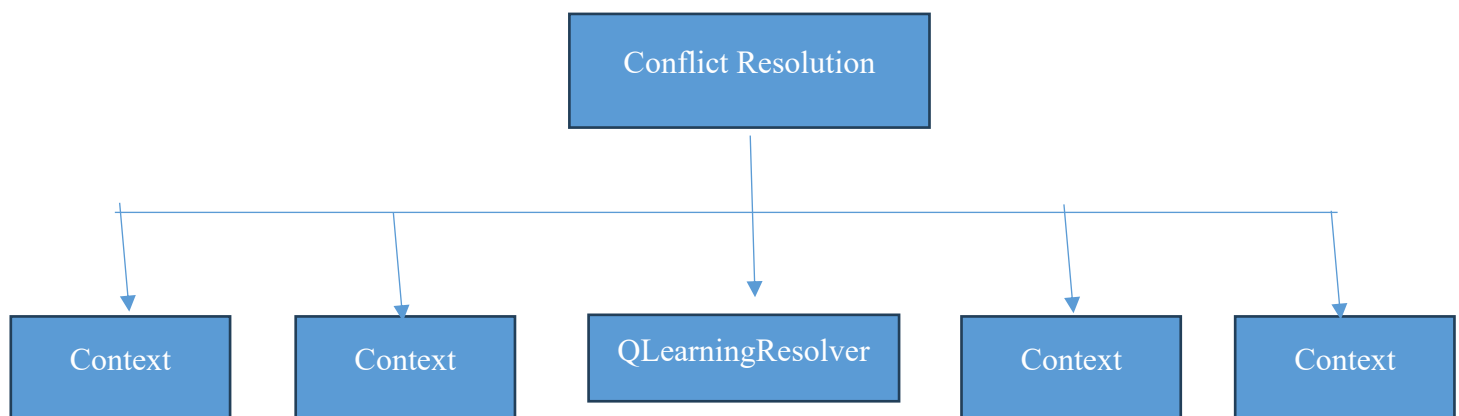


Figura 4.3 - Organização do módulo Conflict Resolution.

Fonte: Elaboração própria.

O ficheiro **base.py** define a interface abstrata **ConflictResolver**, estabelecendo o comportamento comum esperado para qualquer estratégia de resolução de conflitos. O ficheiro **context.py** implementa a estrutura **ConflictContext**, utilizada para representar o estado da simulação no instante em que ocorre um conflito, reunindo informação relevante sobre os robôs

envolvidos e o ambiente circundante. Por sua vez, `actions.py` define o conjunto de ações disponíveis para resolver conflitos, enquanto `rewards.py` implementa a função de recompensa utilizada durante o processo de aprendizagem.

A estratégia baseada em aprendizagem por reforço encontra-se implementada em `q_agent.py`, através da classe **QLearningResolver**. Esta componente utiliza um algoritmo de **Q-Learning** para selecionar a ação mais adequada em cada situação de conflito, recorrendo à experiência adquirida ao longo do processo de treino. Durante a aprendizagem, as ações executadas são avaliadas pela função de recompensa, permitindo atualizar progressivamente a política de decisão utilizada pelo agente.

A separação entre a representação do contexto, a definição das ações, o cálculo das recompensas e a estratégia de decisão contribui para uma arquitetura modular e extensível. Esta organização facilita a integração de novas políticas de resolução de conflitos sem necessidade de alterar os restantes componentes da aplicação, mantendo reduzido o acoplamento entre os módulos do sistema.

4.2.5 Módulo Metrics

O módulo Metrics é responsável pela recolha, processamento e disponibilização das métricas geradas durante a execução da simulação. A sua implementação segue uma abordagem baseada em eventos, na qual a Engine emite acontecimentos relevantes e o módulo de métricas os regista de forma passiva, permitindo avaliar o desempenho do sistema sem interferir na execução dos algoritmos de navegação ou de resolução de conflitos.

A implementação encontra-se distribuída pelos ficheiros **events.py**, **simulation_metrics.py** e **__init__.py**. O ficheiro **events.py** define os diferentes tipos de eventos que podem ocorrer durante a simulação, enquanto **simulation_metrics.py** implementa a classe **SimulationMetrics**, responsável por processar os eventos recebidos e atualizar os indicadores utilizados na avaliação experimental.

Os eventos registados constituem a base para o cálculo das métricas experimentais utilizadas na avaliação do sistema. Os indicadores produzidos por este módulo permitem comparar o desempenho das diferentes estratégias de navegação e resolução de conflitos implementadas, fornecendo os dados que serão posteriormente analisados no Capítulo 6.

Tabela 4.3 - Principais responsabilidades do módulo Metrics.

Fonte: Elaboração própria.

Componente	Responsabilidade
events.py	Define os tipos de eventos utilizados para monitorizar a execução da simulação.
Event	Representa cada evento registado, incluindo o tipo de evento, instante temporal e informação associada.
simulation_metrics.py	Processa os eventos recebidos e atualiza as métricas da simulação.
Indicadores	Calcula métricas como <i>makespan</i> , <i>success rate</i> , colisões, replaneamentos, esperas e tempo médio de conclusão.
Exportação CSV	Disponibiliza os resultados para posterior análise estatística e comparação experimental.

A classe `SimulationMetrics` mantém informação relativa ao progresso da simulação, atualizando continuamente os indicadores sempre que ocorre um novo evento. Entre as métricas calculadas incluem-se o número de colisões, replaneamentos, esperas, robôs que atingem o objetivo, *makespan*, taxa de sucesso (*success rate*), tempo médio de conclusão e número médio de passos executados por robô.

A separação entre a execução da simulação e a avaliação do seu desempenho contribui para uma arquitetura modular e de baixo acoplamento. Enquanto a **Engine** é responsável por controlar a evolução da simulação e emitir os eventos correspondentes, o módulo **Metrics** limita-se a observar esses acontecimentos, calcular os indicadores de desempenho e disponibilizar os resultados para visualização na interface gráfica e posterior exportação em formato CSV. Esta abordagem facilita a comparação entre diferentes estratégias de navegação e resolução de conflitos, constituindo a base da avaliação experimental apresentada no Capítulo 6.

4.2.6 Modulo Viewer

O módulo `Viewer` é responsável pela representação gráfica da simulação e pela interação do utilizador com a aplicação. A sua implementação encontra-se concentrada no ficheiro `viewer.py`, desenvolvido com recurso à biblioteca `Pygame`, permitindo acompanhar visualmente a execução da simulação em tempo real.

Durante a inicialização da aplicação, este módulo procede à criação dos principais componentes necessários à execução da simulação, incluindo a **Engine**, o sistema de métricas

e os robôs definidos para o cenário. Quando a estratégia selecionada corresponde ao **Q-Learning**, o módulo efetua igualmente o carregamento da Q-table previamente treinada, disponibilizando-a ao mecanismo de resolução de conflitos antes do início da execução.

A interface gráfica representa os diferentes elementos do ambiente de simulação, incluindo a grelha, obstáculos, robôs, objetivos e trajetórias calculadas pelos algoritmos de navegação. A atualização contínua destes elementos permite acompanhar a evolução da simulação e observar, de forma intuitiva, o comportamento dos robôs ao longo da execução.

Além da representação gráfica, o módulo disponibiliza um painel informativo (*Heads-Up Display – HUD*) onde são apresentados diversos indicadores relativos ao estado da simulação, incluindo o número de *ticks* executados, robôs concluídos, colisões, conflitos detetados, esperas, replaneamentos, *makespan*, taxa de sucesso e tempo médio de conclusão. Esta informação complementa a visualização do ambiente e permite acompanhar o desempenho da simulação em tempo real.

Embora constitua o ponto de entrada da execução interativa da aplicação, o módulo **Viewer** limita-se à gestão da interface gráfica e da interação com o utilizador, delegando a lógica de execução da simulação para o módulo **Environment**. A aplicação disponibiliza ainda modos de execução não gráfica destinados ao treino do agente de Q-Learning e à realização das experiências utilizadas na avaliação do sistema. Esta separação entre apresentação e lógica de negócio contribui para uma arquitetura modular e facilita a evolução independente dos diferentes componentes da aplicação.

5 Implementação

O presente capítulo descreve a implementação do sistema MAPF Smart Warehouse, apresentando os principais componentes desenvolvidos e a forma como estes foram integrados para permitir a simulação de navegação multi-robô em ambientes logísticos. Enquanto o capítulo anterior apresentou a arquitetura geral da solução, este capítulo foca-se nos detalhes técnicos da implementação, incluindo a representação do ambiente, os agentes, os algoritmos de planeamento, os mecanismos de resolução de conflitos, o sistema de métricas e a interface gráfica.

A implementação foi organizada de forma modular, seguindo a estrutura apresentada no Capítulo 4. Esta organização permitiu separar claramente as responsabilidades de cada componente, facilitando o desenvolvimento incremental do sistema e a posterior integração de

funcionalidades mais avançadas, como o planeamento com A*, a gestão de conflitos entre robôs e a utilização de Q-Learning como apoio à tomada de decisão em situações de bloqueio ou potencial colisão.

Ao longo deste capítulo são descritas as principais decisões de implementação, destacando os ficheiros, classes e funções mais relevantes do projeto. Sempre que necessário, são apresentados excertos de código ou representações simplificadas dos processos implementados, de forma a clarificar o funcionamento interno do sistema sem transformar o relatório numa documentação completa do código-fonte.

5.1 Introdução à Implementação

A implementação do sistema MAPF Smart Warehouse foi organizada segundo a arquitetura modular apresentada no Capítulo 4, permitindo separar claramente as diferentes responsabilidades da aplicação. Esta organização facilitou o desenvolvimento incremental do projeto, tornando possível implementar e testar cada componente de forma independente antes da sua integração no sistema completo.

O desenvolvimento iniciou-se pela implementação da representação do ambiente de simulação e do motor responsável pela coordenação da execução. Posteriormente foram desenvolvidos os agentes, os algoritmos de planeamento de trajetórias, os mecanismos de resolução de conflitos e o sistema de recolha de métricas. Finalmente, foi implementada a interface gráfica, responsável pela visualização da simulação e pela interação com o utilizador.

As secções seguintes descrevem os principais componentes implementados, evidenciando as decisões técnicas adotadas e os aspetos mais relevantes da sua implementação. Sempre que apropriado, são apresentados pequenos excertos de código e diagramas simplificados para ilustrar o funcionamento interno dos diferentes módulos.

5.2 Implementação do Ambiente de Simulação

O ambiente de simulação constitui a base de funcionamento do sistema, sendo responsável pela representação do armazém, gestão dos robôs, controlo dos obstáculos e coordenação da execução da simulação. A sua implementação encontra-se distribuída pelos ficheiros `grid.py`, `loader.py`, `engine.py`, `multi_robot.py` e `obstacles.py`, cada um dedicado a uma funcionalidade específica.

A implementação foi concebida de forma modular, permitindo separar a representação do ambiente da lógica de execução da simulação. Esta abordagem facilita a manutenção do código e possibilita a reutilização dos diferentes componentes em cenários distintos, alterando apenas os mapas utilizados ou os parâmetros de configuração definidos pelo utilizador.

Nas secções seguintes são apresentados os principais componentes que constituem este módulo, descrevendo a forma como o ambiente é representado, como os mapas são carregados e como a execução da simulação é coordenada pelo motor principal.

5.2.1 Representação da Grelha

A representação do ambiente constitui um dos elementos fundamentais da implementação do sistema, servindo de base ao cálculo de trajetórias, deteção de obstáculos e movimentação dos robôs. Para este efeito foi desenvolvida a classe `Grid`, implementada no ficheiro `grid.py`, responsável por armazenar a estrutura do armazém e disponibilizar as operações de consulta utilizadas pelos restantes módulos da aplicação.

A grelha é representada através de uma matriz bidimensional, na qual cada célula assume um dos valores definidos pela enumeração `Cell`: **FREE**, correspondente a uma célula navegável, e **OBSTACLE**, correspondente a um obstáculo estático. A implementação segue a convenção de coordenadas (x, y) , em que x representa a coluna e y a linha, sendo o acesso às células efetuado através da expressão `cells[y][x]`. Esta representação permite armazenar de forma simples e eficiente o estado do ambiente, constituindo a base para todas as operações de navegação realizadas durante a simulação.

```
class Cell(IntEnum):
    FREE = 0
    OBSTACLE = 1

class Grid:
    def __init__(self, width: int, height: int):
        self.width = width
        self.height = height
        self.cells = [
            [Cell.FREE] * width for _ in range(height)
        ]
```

Código 5.1 - Estrutura base da classe `Grid`.

Fonte: Elaboração própria.

A classe disponibiliza um conjunto de métodos responsáveis pela consulta da grelha, permitindo verificar se uma determinada posição pertence aos limites do mapa, confirmar se uma célula é navegável e obter os vizinhos ortogonais acessíveis a partir de uma posição. O método **neighbors()** devolve um iterador sobre as posições caminháveis adjacentes, considerando apenas movimentos horizontais e verticais. Esta abordagem simplifica o processo de exploração efetuado pelo algoritmo A* e reduz o número de estados gerados durante o planeamento das trajetórias.

```
def in_bounds(self, x: int, y: int) -> bool:
    return 0 <= x < self.width and 0 <= y < self.height

def is_walkable(self, x: int, y: int) -> bool:
    return self.in_bounds(x, y) and self.cells[y][x] == Cell.FREE

def neighbors(self, x: int, y: int):
    for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nx, ny = x + dx, y + dy
        if self.is_walkable(nx, ny):
            yield (nx, ny)
```

Código 5.2 - Principais métodos de consulta da classe Grid.

Fonte: Elaboração própria.

Para além das operações de consulta, a classe inclui métodos que permitem alterar o estado das células da grelha, marcando uma posição como obstáculo ou restaurando-a como célula livre. Estas operações são utilizadas durante a construção do ambiente e pelo sistema de gestão de obstáculos estáticos, mantendo a representação da grelha consistente ao longo da execução da simulação. O carregamento dos mapas utilizados pelo sistema é realizado através de um módulo específico, descrito na secção seguinte, responsável por interpretar o formato **MovingAI** e construir automaticamente a estrutura da grelha.

5.2.2 Carregamento dos Mapas

O carregamento dos cenários de simulação é realizado através do ficheiro `loader.py`, responsável por interpretar mapas no formato **MovingAI** e convertê-los para a estrutura de dados utilizada internamente pelo sistema. Esta abordagem permite utilizar cenários normalizados amplamente adotados na investigação em problemas de Multi-Agent Path Finding (MAPF), facilitando a reprodução das experiências e a comparação entre diferentes estratégias de navegação.

A implementação baseia-se na função **`load_map()`**, que recebe o caminho para um ficheiro de mapa e devolve uma instância da classe **`Grid`** devidamente inicializada. Durante este processo, o ficheiro é validado antes da leitura do ambiente, verificando a existência do mapa, a integridade do cabeçalho e a consistência das dimensões especificadas. Caso seja detetada alguma inconsistência, são geradas exceções apropriadas, evitando a utilização de cenários inválidos durante a execução da simulação.

```
def load_map(path: Path) -> Grid:
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(...)

    lines = path.read_text(encoding="utf-8").splitlines()
    ...
```

Código 5.3 – Estrutura inicial da função `load_map()`.

Fonte: Elaboração própria.

Após a validação do cabeçalho, a função extrai as dimensões do mapa e cria uma nova instância da classe **`Grid`**. Em seguida, percorre cada linha da representação textual do ambiente, identificando automaticamente as células correspondentes a obstáculos. Para este efeito, são considerados como obstáculos os caracteres **@**, **O**, **T** e **#**, enquanto os restantes símbolos são interpretados como posições navegáveis. Esta estratégia torna a implementação compatível com diferentes variantes do formato **MovingAI**, sem necessidade de adaptações adicionais.

```
grid = Grid(width, height)

for row, line in enumerate(body):
    for col in range(min(width, len(line))):
        ch = line[col]
        if ch in OBSTACLE_CHARS:
```



```
grid.cells[row][col] = Cell.OBSTACLE
```

Código 5.4 – Construção da grelha a partir do mapa *MovingAI*.

Fonte: Elaboração própria.

A separação entre o carregamento dos mapas e a representação da grelha contribui para uma arquitetura modular, permitindo que novos cenários sejam introduzidos sem alterações aos restantes componentes do sistema. Desta forma, os algoritmos de navegação e os mecanismos de resolução de conflitos operam sempre sobre uma estrutura uniforme, independentemente do ficheiro de origem utilizado para construir o ambiente de simulação.

5.2.3 Motor da Simulação

O controlo da execução da simulação é assegurado pela classe *Engine*, implementada no ficheiro *engine.py*, constituindo o núcleo do sistema desenvolvido. Esta classe é responsável por coordenar a interação entre os diferentes módulos da aplicação, incluindo o ambiente de simulação, os robôs, o sistema de resolução de conflitos e o módulo de métricas, garantindo a evolução consistente da simulação ao longo do tempo.

Durante a inicialização, a **Engine** procede ao carregamento do mapa selecionado através da função **load_map()**, cria o gestor de obstáculos, inicializa as estruturas necessárias ao controlo dos robôs e estabelece as ligações aos módulos responsáveis pela recolha de métricas e pela resolução de conflitos. Esta abordagem centraliza a gestão da simulação numa única classe, reduzindo o acoplamento entre os restantes componentes do sistema.

```
class Engine:

    def __init__(
        self,
        map_path,
        metrics=None,
        conflict_resolver=None,
        reward_fn=default_reward,
    ):
        self.grid = load_map(Path(map_path))
        self.obstacle_manager = ObstacleManager(self.grid)
        self.robots = []
```

```
self.tick_count = 0
```

Código 5.5 - Inicialização da classe Engine.

Fonte: Elaboração própria.

A evolução da simulação é efetuada através do método **step()**, responsável por executar um ciclo completo de atualização. Em cada iteração são realizadas operações como o replaneamento reativo, a determinação dos movimentos pretendidos pelos robôs, a deteção e resolução de conflitos, a movimentação dos agentes, a atualização dos obstáculos dinâmicos, o registo de eventos e a atualização das métricas da simulação. No final de cada ciclo é devolvido um conjunto estruturado de informações relativas ao estado corrente da execução, utilizadas pela interface gráfica para atualizar a representação do ambiente.

```
def step(self) -> dict:
    self.tick_count += 1

    replanned = self._reactive_replanning()
    intents = self._intended_moves()

    ...

    self._execute_movements(...)
    self._detect_collisions(...)
    self._update_metrics(...)

    return self._step_result(...)
```

Código 5.6 - Estrutura simplificada do ciclo principal da simulação.

Fonte: Elaboração própria.

A implementação da **Engine** segue uma abordagem modular, recorrendo a métodos privados especializados para executar cada uma das etapas do ciclo de simulação. Esta organização reduz a complexidade do método principal, melhora a legibilidade do código e facilita a manutenção da aplicação, permitindo que funcionalidades como o replaneamento, a resolução de conflitos, a atualização das métricas e a deteção de colisões sejam desenvolvidas e testadas de forma independente.

5.3 Implementação dos Agentes

5.3.1 Classe Robot

A representação dos agentes móveis é efetuada através da classe `Robot`, implementada no ficheiro `robot.py`. Cada instância desta classe representa um robô autónomo responsável por se deslocar entre uma posição inicial e um objetivo previamente definido, mantendo toda a informação necessária ao planeamento, execução e monitorização da sua trajetória durante a simulação.

Cada robô armazena informação relativa ao seu identificador, posição inicial, posição de destino e posição atual, bem como o caminho planeado, o número de passos executados, o número de replaneamentos efetuados e o estado da execução. Esta informação é continuamente atualizada durante a simulação, permitindo acompanhar o progresso individual de cada agente e recolher indicadores utilizados posteriormente na avaliação do sistema.

```
class Robot:
    def __init__(self, robot_id, start, goal):
        self.robot_id = robot_id
        self.start = start
        self.goal = goal
        self.current_position = start

        self.path = []
        self._path_index = 0
```

Código 5.7 - Estrutura base da classe Robot.

Fonte: Elaboração própria.

O planeamento das trajetórias é realizado através do método `plan_path()`, responsável por selecionar automaticamente o algoritmo de navegação configurado e calcular um percurso entre a posição atual e o objetivo do robô. A implementação permite utilizar diferentes estratégias de planeamento através da função `_select_planner()`, mantendo uma interface comum independentemente do algoritmo utilizado. Esta abordagem reduz o acoplamento entre a classe **Robot** e os algoritmos de navegação, facilitando a introdução de novas estratégias sem necessidade de alterar a restante implementação.

```
def plan_path(self, grid, max_time=200, **kwargs):
    selected_planner = _select_planner()
    path = selected_planner(
```

```

        grid,
        self.current_position,
        self.goal,
        max_time=max_time,
        **kwargs,
    )

```

Código 5.8 - Planeamento de trajetórias através do método `plan_path()`.

Fonte: Elaboração própria.

Após o cálculo do percurso, o robô executa a navegação de forma incremental, avançando uma posição em cada *tick* da simulação através do método `move_one_step()`. Esta abordagem permite que o motor da simulação acompanhe continuamente o estado de cada agente, possibilitando a deteção de conflitos, o replaneamento de trajetórias e a atualização das métricas sempre que necessário. Sempre que um robô atinge o seu objetivo, o seu estado é atualizado, deixando de participar nas restantes iterações da simulação.

A implementação inclui ainda métodos auxiliares para consultar o estado do agente, verificar se existe um caminho disponível, efetuar replaneamentos e reinicializar completamente o robô para novas execuções experimentais. Estas funcionalidades simplificam a integração da classe com o motor da simulação e contribuem para uma implementação modular e reutilizável.

5.3.2 Implementação do Algoritmo A*

O algoritmo de navegação principal do sistema foi implementado no ficheiro `a_star.py`, recorrendo a uma extensão temporal do algoritmo A*. Ao contrário da implementação clássica, em que cada estado representa apenas uma posição da grelha, a solução desenvolvida representa cada estado através do par $((x, y), t)$, onde (x, y) identifica a posição do robô e t corresponde ao instante temporal da simulação. Esta representação permite incorporar explicitamente a evolução temporal durante o planeamento das trajetórias, constituindo a base para a coordenação entre múltiplos robôs.

A função principal `a_star()` recebe como parâmetros a grelha de navegação, a posição inicial, a posição objetivo e um horizonte temporal máximo, suportando ainda conjuntos opcionais de restrições sobre vértices e arestas. Estas restrições permitem impedir que um robô ocupe determinadas posições em instantes específicos ou realize transições proibidas entre células consecutivas, possibilitando a integração do algoritmo com mecanismos cooperativos de planeamento e resolução de conflitos.

```
def a_star(
    grid,
    start,
    goal,
    max_time=200,
    vertex_constraints=None,
    edge_constraints=None,
):
```

*Código 5.9 - Definição da função principal do algoritmo A.**

Fonte: Elaboração própria.

A exploração do espaço de estados é efetuada através de uma fila de prioridade implementada com a biblioteca **heapq**, na qual os estados são ordenados segundo o valor da função de avaliação $f(n)=g(n)+h(n)$. O custo acumulado $g(n)$ corresponde ao número de movimentos efetuados desde a posição inicial, enquanto a heurística $h(n)$ utiliza a distância de Manhattan entre a posição atual e o objetivo. Esta heurística é admissível para movimentos ortogonais numa grelha 4-conectada, garantindo que o algoritmo preserve as propriedades de completude e otimalidade.

```
def _heuristic(pos, goal):
    return abs(pos[0] - goal[0]) + abs(pos[1] - goal[1])
```

Código 5.10 - Heurística de Manhattan utilizada pelo algoritmo.

Fonte: Elaboração própria.

A geração dos estados sucessores é realizada através da função **_neighbors()**, responsável por produzir os quatro movimentos ortogonais possíveis e uma ação adicional de permanência na posição atual (*wait*). Esta última ação desempenha um papel importante em ambientes multi-robô, permitindo que um agente permaneça temporariamente parado para evitar conflitos ou satisfazer restrições temporais impostas durante o planeamento.

```
def _neighbors(state):
    (x, y), t = state
    moves = ((0, 1), (1, 0), (0, -1), (-1, 0), (0, 0))
    return [(x + dx, y + dy), t + 1 for dx, dy in moves]
```

Código 5.11 - Geração dos estados sucessores.

Fonte: Elaboração própria.

Durante o processo de expansão, cada estado gerado é validado antes de ser inserido na fila de prioridade. São verificadas as condições de navegabilidade da célula, o cumprimento das

restrições temporais sobre vértices e arestas e a existência de um caminho de menor custo previamente conhecido. Sempre que um estado atinge a posição objetivo, o percurso completo é reconstruído através da estrutura `came_from`, devolvendo uma sequência ordenada de estados espaço-temporais que será posteriormente utilizada pelo robô durante a execução da simulação.

A implementação foi concebida de forma modular, distribuindo funcionalidades específicas por métodos auxiliares responsáveis pelo cálculo da heurística, geração dos vizinhos, validação das posições e reconstrução da trajetória final. Esta organização melhora a legibilidade do código, facilita a manutenção da implementação e permite reutilizar os diferentes componentes em algoritmos cooperativos apresentados nas secções seguintes.

5.3.3 Planeamento Multi-Robô

O suporte ao planeamento em cenários com múltiplos robôs é implementado através do ficheiro `prioritized_a_star.py`, cuja principal função consiste em disponibilizar uma interface compatível com os restantes algoritmos de navegação utilizados pelo sistema. Ao contrário de implementações tradicionais de Prioritized A*, este módulo não é responsável pela coordenação direta entre os robôs, limitando-se a encaminhar os parâmetros de planeamento para a implementação do algoritmo A* descrita na secção anterior.

A coordenação entre agentes é efetuada pelo motor da simulação (**Engine**), responsável por construir e atualizar as restrições temporais utilizadas durante o planeamento. Sempre que necessário, a **Engine** fornece conjuntos de restrições sobre vértices e arestas ao algoritmo A*, impedindo que um robô planeie trajetórias incompatíveis com os percursos já atribuídos aos restantes agentes. Esta abordagem permite reutilizar a mesma implementação do algoritmo A* tanto em cenários individuais como em cenários cooperativos, evitando a duplicação de código.

```
def plan(
    grid,
    start,
    goal,
    max_time=200,
    vertex_constraints=None,
    edge_constraints=None,
):
    return a_star(
        grid,
        start,
```

```

        goal,
        max_time=max_time,
        vertex_constraints=vertex_constraints,
        edge_constraints=edge_constraints,
    )

```

Código 5.12 - Interface do módulo `prioritized_a_star.py`.

Fonte: Elaboração própria.

A separação entre o algoritmo de navegação e o mecanismo responsável pela coordenação dos robôs contribui para uma arquitetura modular e extensível. Enquanto o módulo **Agents** permanece responsável pelo cálculo das trajetórias individuais, o controlo das prioridades, a deteção de conflitos, os replaneamentos e a aplicação das restrições temporais são efetuados pela **Engine** e pelos mecanismos de resolução de conflitos descritos na secção seguinte. Esta organização facilita a manutenção do sistema e permite integrar diferentes estratégias de coordenação sem necessidade de alterar a implementação do algoritmo de navegação.

5.4 Implementação da Resolução de Conflitos

5.4.1 Contexto de Conflito

O processo de resolução de conflitos baseia-se na classe `ConflictContext`, implementada no ficheiro `context.py`, responsável por representar toda a informação necessária para a tomada de decisão quando ocorre um conflito entre robôs. Em vez de o mecanismo de resolução aceder diretamente ao estado interno da simulação, todas as informações relevantes são agregadas numa única estrutura de dados, simplificando a comunicação entre o motor da simulação e o módulo de decisão.

A implementação recorre ao decorador `@dataclass` da linguagem Python, permitindo definir uma estrutura imutável (*frozen*) composta apenas pelos atributos necessários ao processo de decisão. Entre estes atributos encontram-se o instante temporal da simulação, o identificador do robô, a posição atual, a posição pretendida, o objetivo, o número de *ticks* consecutivos em que o robô permaneceu bloqueado e a informação relativa aos restantes robôs envolvidos na situação de conflito.

```

@dataclass(frozen=True)
class ConflictContext:
    tick: int
    robot_id: int

```

```
current_position: Position
intended_position: Position
goal: Position
blocked_ticks: int = 0
```

Código 5.13 – Estrutura simplificada da classe ConflictContext.

Fonte: Elaboração própria.

A utilização de uma estrutura imutável garante que o estado observado pelo mecanismo de decisão permanece consistente durante todo o processo de resolução do conflito, evitando alterações acidentais da informação utilizada pelos algoritmos de decisão. Adicionalmente, a classe inclui um campo genérico destinado a futuras extensões, permitindo incorporar novos atributos sem comprometer a compatibilidade com os restantes componentes da aplicação.

Esta representação constitui igualmente a observação utilizada pelo agente de Q-Learning, fornecendo a informação necessária para selecionar a ação mais adequada em cada situação de conflito. A separação entre a representação do contexto e os algoritmos de decisão contribui para uma arquitetura modular, permitindo substituir ou acrescentar novas estratégias de resolução sem necessidade de alterar a estrutura da informação disponibilizada pelo sistema.

5.4.2 Ações e Sistema de Recompensas

A tomada de decisão durante a resolução de conflitos baseia-se num conjunto reduzido de ações definidas no ficheiro `actions.py`. Estas ações representam as possíveis decisões que podem ser adotadas quando um robô se encontra numa situação de conflito, fornecendo uma interface comum para os diferentes mecanismos de decisão implementados no sistema. A utilização de uma enumeração garante uma representação consistente das ações disponíveis e simplifica a integração com o agente de aprendizagem por reforço.

Na implementação atual encontram-se definidas três ações: **WAIT**, que mantém o robô na posição atual durante um *tick* da simulação; **YIELD**, que permite ao robô ceder passagem através de um desvio quando possível; e **REPLAN**, utilizada para solicitar o recálculo da trajetória. Embora as três ações permaneçam disponíveis na interface do sistema, a política de aprendizagem utilizada neste projeto centra-se essencialmente nas decisões **WAIT** e **YIELD**, sendo o replaneamento tratado pela **Engine** como mecanismo de recuperação em situações persistentes de bloqueio.

```
class ConflictAction(str, Enum):
    WAIT = "wait"
    YIELD = "yield"
```



```
REPLAN = "replan"
```

Código 5.14 - Ações disponíveis para a resolução de conflitos.

Fonte: Elaboração própria.

A avaliação das decisões tomadas pelo agente é efetuada através da função `default_reward()`, implementada no ficheiro `rewards.py`. Esta função calcula uma recompensa numérica com base no contexto anterior e posterior à ação executada, incentivando comportamentos que favorecem a progressão dos robôs e penalizando decisões que conduzem a colisões, bloqueios prolongados ou afastamento do objetivo.

A função de recompensa atribui uma penalização elevada sempre que ocorre uma colisão e uma recompensa significativa quando o robô atinge o seu objetivo. Para as restantes situações, a recompensa é calculada considerando a evolução da distância de Manhattan até ao destino, bem como o número de *ticks* consecutivos em que o robô permanece bloqueado. Desta forma, decisões que contribuem para o progresso da navegação tendem a ser favorecidas, enquanto ações que aumentam o congestionamento ou prolongam estados de bloqueio recebem uma penalização proporcional.

```
if collided:
    return -150.0
if reached_goal:
    return 80.0
```

Código 5.15 - Recompensas associadas aos estados terminais.

Fonte: Elaboração própria.

A implementação procura manter uma política de aprendizagem conservadora, atribuindo uma penalização reduzida à ação `WAIT` quando utilizada durante curtos períodos e recompensando a ação `YIELD` apenas quando esta conduz efetivamente a progresso na navegação. Esta estratégia reduz a adoção de desvios desnecessários e privilegia decisões que contribuam para uma resolução eficiente dos conflitos, mantendo simultaneamente a estabilidade da aprendizagem do agente.

5.4.3 Implementação do Q-Learning

A aprendizagem por reforço foi implementada através da classe `QLearningResolver`, definida no ficheiro `q_agent.py`. Ao contrário de abordagens em que o agente substitui completamente o algoritmo de navegação, nesta implementação o Q-Learning atua apenas

como um mecanismo auxiliar de decisão local durante situações de conflito. O cálculo das trajetórias continua a ser realizado pelo algoritmo A*, enquanto o replaneamento global permanece sob responsabilidade da Engine. O agente de aprendizagem intervém apenas na escolha da ação mais adequada quando um robô fica temporariamente bloqueado por outro agente.

O estado observado pelo agente é obtido a partir da classe **ConflictContext**, sendo posteriormente convertido para uma representação discreta através da função **encode_state()**. Este processo reduz a dimensionalidade do problema, agrupando informação como a direção relativa ao objetivo, a distância de Manhattan, o tempo de bloqueio, o nível de congestionamento local e o alinhamento com o destino. A discretização do estado permite utilizar uma Q-table compacta, adequada à aprendizagem tabular implementada neste projeto.

```
State = (  
    dx_sign,  
    dy_sign,  
    intended_dx_sign,  
    intended_dy_sign,  
    dist_bucket,  
    blocked_bucket,  
    congestion_bucket,  
    at_goal_axis,  
)
```

Código 5.16 – Representação discreta do estado utilizada pelo agente.

Fonte: Elaboração própria.

A seleção das ações segue uma política ϵ -greedy, permitindo equilibrar exploração e exploração durante a fase de treino. Sempre que o agente se encontra em modo de aprendizagem, uma ação pode ser escolhida aleatoriamente com probabilidade ϵ ; caso contrário, é selecionada a ação com maior valor esperado na Q-table. Durante a fase de avaliação, o valor de ϵ é automaticamente definido para zero, garantindo uma política puramente determinística.

```
if self.training and self._rng.random() < self.epsilon:  
    action_idx = self._rng.choice(allowed)  
else:  
    action_idx = self._select_greedy_action(state, allowed)
```

Código 5.17 - Seleção de ações segundo a política ϵ -greedy.

Fonte: Elaboração própria.

Após a execução de cada ação, a função **observe()** procede à atualização da Q-table recorrendo à equação clássica do algoritmo Q-Learning, utilizando a recompensa calculada pelo sistema apresentado na secção anterior e o maior valor esperado para o estado seguinte. Desta forma, o agente ajusta progressivamente a qualidade das suas decisões com base na experiência acumulada ao longo das diferentes simulações.

```
new_q = old_q + self.alpha * (
    reward + self.gamma * future - old_q
)
```

Código 5.18 - Atualização dos valores da Q-table.

Fonte: Elaboração própria.

A implementação inclui ainda mecanismos de persistência que permitem guardar e carregar a Q-table em formato JSON, possibilitando reutilizar conhecimento previamente adquirido sem necessidade de repetir todo o processo de treino. Para garantir a compatibilidade entre diferentes versões da política de aprendizagem, a aplicação valida automaticamente a versão da tabela carregada antes da sua utilização. Adicionalmente, são mantidos contadores estatísticos relativos ao número de decisões, atualizações da Q-table e frequência de utilização de cada ação, facilitando a análise do processo de aprendizagem e a avaliação experimental apresentada no Capítulo 6.

Finalmente, importa salientar que a política de aprendizagem adotada foi concebida de forma conservadora. Em vez de substituir os mecanismos determinísticos já existentes, o agente aprende apenas a decidir entre **esperar** (*WAIT*) e **ceder passagem** (*YIELD*) em situações de conflito local. O replaneamento continua a ser efetuado pela **Engine** como mecanismo de recuperação em situações persistentes de bloqueio, permitindo combinar a robustez do planeamento determinístico com a capacidade adaptativa proporcionada pela aprendizagem por reforço.

5.5 Implementação do Sistema de Métricas

A avaliação do desempenho do sistema é suportada pelo módulo Metrics, constituído pelos ficheiros `events.py` e `simulation_metrics.py`. A implementação segue uma abordagem baseada em eventos, na qual a Engine constitui a única fonte de informação sobre a evolução da simulação, enquanto o módulo de métricas atua exclusivamente como observador, sem interferir na execução dos algoritmos de navegação ou na resolução de conflitos. Esta separação reduz o acoplamento entre componentes e facilita a recolha consistente de dados experimentais.

A representação dos acontecimentos da simulação é efetuada através da classe **Event**, implementada como uma **dataclass** imutável. Cada evento contém o tipo de ocorrência, o instante temporal em que foi registado, o identificador do robô envolvido (quando aplicável) e um conjunto opcional de informações adicionais. A utilização de eventos estruturados permite uniformizar o registo das diferentes operações realizadas durante a simulação e facilita a posterior análise dos resultados.

```
class Event:
    type: EventType
    tick: int
    robot_id: Optional[Any] = None
    payload: Mapping[str, Any]
```

Código 5.19 - Estrutura simplificada da classe Event.

Fonte: Elaboração própria.

A agregação das métricas é efetuada pela classe **SimulationMetrics**, responsável por processar os eventos emitidos pela **Engine** e atualizar, de forma incremental, os indicadores estatísticos da simulação. Sempre que um novo evento é recebido através do método **record()**, são atualizados automaticamente os contadores de colisões, replaneamentos, esperas e restantes indicadores necessários para a avaliação do desempenho do sistema. Esta abordagem evita o processamento repetido do histórico completo de eventos, permitindo que as métricas estejam permanentemente disponíveis ao longo da execução da simulação.

```
def record(self, event: Event) -> None:
    if event.type is EventType.ROBOT_WAIT:
        self.wait_count += 1
    elif event.type is EventType.ROBOT_REPLAN:
        self.replan_count += 1
    elif event.type is EventType.COLLISION:
        self.collision_count += 1
```

Código 5.20 - Atualização incremental das métricas a partir dos eventos.

Fonte: Elaboração própria.

Para além dos contadores básicos, a implementação calcula automaticamente diversas métricas derivadas, incluindo o makespan, a taxa de sucesso, o tempo médio de conclusão, o throughput, a taxa de replaneamento, o número médio de *ticks* bloqueados por robô e o número médio de passos executados. Estes indicadores são disponibilizados através de propriedades da

classe, permitindo o seu acesso direto pelos módulos responsáveis pela execução das experiências e pela geração dos resultados apresentados no Capítulo 6.

Por fim, o método `to_dict()` converte todas as métricas recolhidas para uma estrutura de dados normalizada, posteriormente utilizada pelo *runner* experimental para gerar os ficheiros CSV que suportam a análise quantitativa realizada neste trabalho. Esta separação entre a recolha das métricas e a exportação dos resultados contribui para uma implementação modular e facilita a reutilização do sistema em diferentes cenários experimentais.

5.6 Implementação da Interface Gráfica

A interação entre o utilizador e o sistema é assegurada pelo ficheiro `viewer.py`, responsável pela implementação da interface gráfica desenvolvida com a biblioteca Pygame. Este módulo coordena a inicialização da simulação, a criação da janela gráfica e a atualização contínua da representação visual do ambiente, permitindo acompanhar em tempo real a execução dos algoritmos de navegação e resolução de conflitos.

Durante a inicialização, a interface cria uma instância da **Engine**, inicializa o módulo de métricas e gera os robôs que irão participar na simulação. Quando o algoritmo selecionado corresponde ao modo de aprendizagem por reforço, a interface procede ainda ao carregamento da Q-table previamente treinada, permitindo executar a simulação em modo de avaliação sem necessidade de novo treino. Esta lógica encontra-se concentrada na função `build_engine()`, responsável por preparar todos os componentes necessários antes do início da execução.

```
def build_engine():
    metrics = SimulationMetrics()
    ...
    engine = Engine(...)
    robots = create_robots(...)
    engine.start(...)
    return engine, robots, metrics
```

Código 5.21 – Inicialização dos principais componentes da simulação.

Fonte: Elaboração própria.

Após a inicialização, a aplicação entra num ciclo principal de execução responsável por processar os eventos do utilizador, atualizar o estado da simulação e redesenhar continuamente o ambiente gráfico. A evolução temporal da simulação é controlada através de chamadas periódicas ao método `step()` da **Engine**, garantindo a sincronização entre o estado interno do

sistema e a informação apresentada ao utilizador. Durante este ciclo são igualmente tratados comandos como pausa, reinício da simulação e encerramento da aplicação.

```
if not paused and now - last_move_time >= config.MOVE_DELAY:  
    last_state = engine.step()
```

Código 5.22 – Atualização periódica da simulação.

Fonte: Elaboração própria.

A interface representa graficamente os diferentes elementos do ambiente, incluindo a grelha de navegação, obstáculos estáticos e dinâmicos, posições inicial e final de cada robô, trajetórias calculadas, posição atual dos agentes e locais onde ocorrem colisões. Paralelamente, é apresentado um painel informativo (*HUD*) que disponibiliza, em tempo real, indicadores como o número de robôs concluídos, colisões, replaneamentos, conflitos evitados, *makespan*, taxa de sucesso e algoritmo atualmente em execução. Estas informações são obtidas diretamente da Engine e do módulo Metrics, proporcionando ao utilizador uma visão detalhada da evolução da simulação.

A implementação da interface gráfica foi desenvolvida de forma independente da lógica de navegação e dos algoritmos de decisão. Esta separação permite que a visualização funcione apenas como uma camada de apresentação, enquanto toda a lógica da simulação permanece concentrada nos módulos descritos nas secções anteriores. Consequentemente, torna-se possível alterar a interface gráfica ou executar o sistema em modo não gráfico (*headless*) sem modificar os restantes componentes da aplicação.

6 Avaliação Experimental

6.1 Metodologia Experimental

A avaliação experimental teve como objetivo analisar o desempenho do sistema desenvolvido em diferentes cenários de navegação multi-robô, comparando três abordagens distintas: o algoritmo A* sem mecanismos de coordenação, o Prioritized A* com coordenação determinística e a abordagem baseada em Q-Learning para apoio à resolução local de conflitos. Esta comparação permitiu avaliar não apenas a capacidade dos robôs concluírem as suas trajetórias, mas também a segurança e eficiência de cada estratégia.

Para garantir uma análise mais completa, a avaliação considerou métricas relacionadas com conclusão, segurança, tempo de execução e coordenação entre robôs. Entre os indicadores analisados encontram-se a taxa de conclusão, o número de colisões, o *makespan*, o tempo médio

de conclusão, o número de esperas, o número de replaneamentos e a taxa de sucesso seguro. Esta última métrica, designada **safe success**, considera uma execução bem-sucedida apenas quando todos os robôs atingem o objetivo sem que ocorram colisões durante a simulação.

A utilização desta métrica é particularmente importante, uma vez que uma execução em que todos os robôs chegam ao destino não deve ser considerada totalmente bem-sucedida caso tenham ocorrido colisões ao longo do percurso. Assim, a avaliação não se limita a medir a rapidez da navegação, mas também a capacidade de cada abordagem manter a segurança do sistema em ambientes com múltiplos agentes.

As experiências foram executadas em campanhas repetidas, utilizando as mesmas quantidades de robôs e a mesma sequência de *seeds* para os diferentes algoritmos avaliados. Esta estratégia permitiu reduzir o impacto de variações aleatórias e tornar a comparação entre abordagens mais justa. Os resultados obtidos foram registados automaticamente em ficheiros CSV, permitindo a análise quantitativa apresentada nas secções seguintes.

6.2 Configuração das Experiências

As experiências foram realizadas recorrendo a uma campanha experimental concebida para comparar, em condições equivalentes, o desempenho das três abordagens implementadas no sistema: A*, Prioritized A* e Q-Learning. Para garantir a comparabilidade dos resultados, todos os algoritmos foram executados utilizando o mesmo ambiente de simulação, os mesmos cenários e a mesma sequência de *seeds*, permitindo reduzir a influência da aleatoriedade na avaliação experimental.

Cada campanha experimental foi composta por **180 execuções independentes**, resultantes da combinação de seis quantidades distintas de robôs (10, 20, 30, 40, 45 e 50) com um conjunto de 30 *seeds* consecutivas. As mesmas *seeds* foram reutilizadas para todos os algoritmos avaliados, assegurando que cada abordagem fosse testada exatamente nas mesmas condições.

Durante todas as experiências foi utilizado o mesmo mapa de navegação, mantendo igualmente constantes o número inicial de obstáculos dinâmicos e o limite máximo de *ticks* permitido para cada execução. Desta forma, qualquer diferença observada nos resultados pode ser atribuída ao comportamento dos algoritmos de navegação e resolução de conflitos, e não a alterações nas condições experimentais.

Tabela 6.1 - Configuração da campanha experimental.

Fonte: Elaboração própria.

Parâmetro	Valor
Algoritmos avaliados	A*, Prioritized A* e Q-Learning
Número de execuções por algoritmo	180
Número de <i>seeds</i>	30 (33–62)
Quantidade de robôs	10, 20, 30, 40, 45 e 50
Obstáculos dinâmicos	10
Número máximo de <i>ticks</i>	1000
Ficheiros de resultados	CSV gerados automaticamente

No caso do **Q-Learning**, o treino e a avaliação foram realizados em fases distintas. O processo de aprendizagem foi efetuado previamente, produzindo uma Q-table persistente, enquanto a campanha experimental utilizou essa tabela em modo de avaliação (*greedy*), com $\epsilon = 0$, impedindo que novas ações exploratórias influenciassem os resultados obtidos. Esta separação garante que os resultados apresentados refletem exclusivamente o desempenho da política aprendida e não o processo de treino.

6.3 Comparação entre os Algoritmos

Nesta secção são apresentados e comparados os resultados obtidos pelas três abordagens implementadas no sistema: A*, Prioritized A* e Q-Learning. A análise baseia-se nos resultados médios obtidos ao longo das 180 execuções realizadas para cada algoritmo, permitindo avaliar o impacto das diferentes estratégias de coordenação no desempenho global do sistema. A comparação considera simultaneamente indicadores de eficiência, como o *makespan*, e indicadores de segurança, como o número de colisões e a taxa de safe success, proporcionando uma avaliação equilibrada entre rapidez de execução e robustez da navegação.

Para facilitar a interpretação dos resultados, a análise encontra-se organizada por métricas de desempenho. Em cada caso são apresentados os valores médios obtidos para as diferentes quantidades de robôs, permitindo observar a evolução do comportamento dos algoritmos à medida que aumenta a densidade do ambiente.

6.3.1 Taxa de Conclusão

Tabela 6.2 - Taxa média de conclusão para os diferentes algoritmos.

Fonte: Elaboração própria.

Robôs	A*	Prioritized A*	Q-Learning
10	1.000	0.993	1.000

20	1.000	0.990	1.000
30	1.000	0.987	1.000
40	1.000	0.985	1.000
45	1.000	0.988	1.000
50	1.000	0.975	1.000

6.3.2 Taxa de Sucesso Seguro

Tabela 6.3 - Taxa média de sucesso seguro.

Fonte: Elaboração própria.

Robôs	A*	Prioritized A*	Q-Learning
10	0.400	0.967	1.000
20	0.000	0.800	0.900
30	0.000	0.600	0.733
40	0.000	0.300	0.233
45	0.000	0.233	0.367
50	0.000	0.033	0.233

6.3.3 Número Médio de Colisões

Tabela 6.4 - Número médio de colisões por execução.

Fonte: Elaboração própria.

Robôs	A*	Prioritized A*	Q-Learning
10	0.87	0.00	0.00
20	19.93	0.13	0.10
30	37.67	0.27	0.33
40	69.33	1.13	1.87
45	101.87	2.67	1.87
50	99.27	4.50	2.70

6.3.4 Makespan Médio

Tabela 6.5 - Makespan médio obtido para cada algoritmo

. Fonte: Elaboração própria.

Robôs	A*	Prioritized A*	Q-Learning
10	186.3	213.9	186.5
20	191.3	283.7	194.3
30	194.8	367.9	256.3
40	195.5	434.5	207.4
45	194.0	405.1	209.9
50	195.6	643.7	288.2

6.4 Discussão dos Resultados

Os resultados experimentais evidenciam diferenças significativas entre as três abordagens avaliadas, demonstrando que cada algoritmo apresenta vantagens e limitações distintas consoante a densidade do ambiente. A análise conjunta das métricas de conclusão, segurança e eficiência permite compreender o impacto dos diferentes mecanismos de coordenação implementados neste projeto.

O algoritmo **A*** apresentou uma taxa de conclusão próxima de 100% em todos os cenários avaliados, confirmando a sua elevada capacidade para encontrar trajetórias individuais. Contudo, a ausência de mecanismos de coordenação entre robôs resultou num aumento muito acentuado do número de colisões à medida que a densidade do ambiente aumentou. Consequentemente, a métrica de **safe success** diminuiu rapidamente, demonstrando que, apesar de eficiente do ponto de vista do planeamento individual, esta abordagem não é adequada para cenários multi-robô com elevada interação entre agentes.

O **Prioritized A*** reduziu drasticamente o número de colisões relativamente ao **A***, evidenciando a eficácia da introdução de restrições temporais e da coordenação determinística entre robôs. No entanto, esta melhoria na segurança foi acompanhada por um aumento do *makespan*, do número de esperas e dos replaneamentos, sobretudo para cenários com maior número de robôs. Estes resultados mostram que a coordenação determinística melhora significativamente a segurança, mas introduz um custo adicional na eficiência global da navegação.

A abordagem baseada em **Q-Learning** apresentou um comportamento intermédio entre as duas soluções anteriores. Em comparação com o A*, conseguiu reduzir significativamente o número de colisões, mantendo simultaneamente uma taxa de conclusão muito elevada. Relativamente ao Prioritized A*, verificou-se uma redução do *makespan* em vários cenários, refletindo uma maior capacidade de adaptação em situações de conflito local. No entanto, os resultados também mostram que a vantagem do Q-Learning não é uniforme para todas as densidades de robôs, verificando-se cenários em que a taxa de **safe success** é inferior à obtida pelo método priorizado.

De uma forma geral, os resultados confirmam que a arquitetura híbrida adotada neste projeto constitui uma solução equilibrada. O algoritmo A* permanece responsável pelo planeamento das trajetórias, enquanto a **Engine** assegura a coordenação global da simulação e o agente de **Q-Learning** intervém apenas na resolução local de conflitos. Esta separação de responsabilidades permitiu combinar a robustez do planeamento determinístico com a capacidade adaptativa da aprendizagem por reforço, obtendo um compromisso entre eficiência e segurança sem substituir os mecanismos clássicos de navegação.

6.5 Limitações da Avaliação Experimental

Embora a campanha experimental tenha permitido comparar de forma consistente as diferentes abordagens implementadas, existem algumas limitações que devem ser consideradas na interpretação dos resultados.

Em primeiro lugar, as experiências foram realizadas utilizando um conjunto específico de mapas, configurações de obstáculos e densidades de robôs. Apesar de terem sido utilizadas diferentes quantidades de agentes e várias *seeds*, os resultados obtidos refletem apenas os cenários considerados neste trabalho e não podem ser generalizados automaticamente para outros ambientes ou configurações de navegação.

Outra limitação prende-se com a geração dos obstáculos dinâmicos. Embora a campanha experimental reutilize a mesma sequência de *seeds* para os diferentes algoritmos, a colocação dos obstáculos dinâmicos não é totalmente determinada pelo parâmetro de *seed* utilizado na criação dos robôs. Consequentemente, pequenas diferenças na distribuição dos obstáculos podem influenciar o comportamento observado em algumas execuções.

Importa igualmente referir que a comparação entre algoritmos foi baseada sobretudo em valores médios das métricas recolhidas. Embora estas médias permitam caracterizar o comportamento global de cada abordagem, uma análise estatística mais aprofundada,

recorrendo por exemplo a intervalos de confiança ou testes de significância, permitiria avaliar com maior rigor se as diferenças observadas são estatisticamente relevantes.

Por fim, o método **Prioritized A*** utilizado neste projeto corresponde a uma abordagem heurística de coordenação entre robôs, não garantindo optimalidade nem completude global. Da mesma forma, o agente de **Q-Learning** foi concebido para atuar apenas na resolução local de conflitos, complementando o algoritmo de planeamento e não substituindo os mecanismos determinísticos existentes. Apesar destas limitações, os resultados obtidos demonstram que a arquitetura proposta constitui uma solução robusta para a navegação multi-robô, fornecendo uma base sólida para futuras extensões e melhorias do sistema.

7 Conclusões e Trabalho Futuro

7.1 Conclusões

O presente projeto teve como objetivo desenvolver e avaliar um sistema de navegação multi-robô capaz de planejar trajetórias, coordenar múltiplos agentes e resolver conflitos de forma eficiente e segura. Para esse efeito, foi implementada uma arquitetura modular composta pelos módulos Environment, Agents, Conflict Resolution, Metrics e Viewer, permitindo separar as diferentes responsabilidades do sistema e facilitar a sua manutenção e evolução.

O algoritmo **A*** foi utilizado como base para o planeamento das trajetórias individuais, enquanto a coordenação global da simulação foi assegurada pela **Engine** através da aplicação de restrições temporais, deteção de conflitos e mecanismos de replaneamento. Sobre esta base foi integrado um agente de **Q-Learning**, concebido para atuar exclusivamente na resolução local de conflitos, complementando os mecanismos determinísticos já existentes em vez de os substituir.

Os resultados experimentais demonstraram que a abordagem proposta constitui um compromisso equilibrado entre eficiência e segurança. O algoritmo **A*** apresentou elevada capacidade de conclusão das trajetórias, mas revelou limitações em cenários com elevada densidade de robôs devido ao aumento significativo do número de colisões. A abordagem **Prioritized A*** reduziu substancialmente essas colisões, embora à custa de um aumento do *makespan* e do número de replaneamentos. Por sua vez, a integração do **Q-Learning** permitiu melhorar a resolução local de conflitos e reduzir o *makespan* em diversos cenários, mantendo simultaneamente níveis de segurança superiores aos obtidos pelo **A*** simples. Estes resultados

confirmam que a utilização da aprendizagem por reforço como mecanismo complementar constitui uma solução viável para melhorar o desempenho global da navegação multi-robô.

Em termos gerais, considera-se que os objetivos inicialmente definidos para o projeto foram alcançados. O sistema desenvolvido demonstra a viabilidade da integração entre algoritmos clássicos de planeamento e técnicas de aprendizagem por reforço, disponibilizando uma plataforma modular que poderá servir de base para trabalhos futuros na área da navegação cooperativa e dos sistemas multi-agente.

7.2 Trabalho Futuro

Apesar dos resultados obtidos, existem diversas possibilidades de evolução do sistema desenvolvido. Uma primeira linha de trabalho consiste na implementação de algoritmos de coordenação mais avançados, como Conflict-Based Search (CBS) ou variantes do MAPF, permitindo comparar o seu desempenho com a abordagem atualmente implementada.

Outra evolução possível consiste no enriquecimento do agente de aprendizagem por reforço. A utilização de técnicas como **Deep Q-Networks (DQN)** ou outros métodos baseados em redes neurais poderá permitir representar estados mais complexos e melhorar a capacidade de adaptação em cenários de elevada densidade ou com ambientes de maiores dimensões.

Também seria interessante reforçar a campanha experimental através da utilização de um maior número de mapas, diferentes distribuições de obstáculos e análises estatísticas mais completas, recorrendo, por exemplo, a intervalos de confiança e testes de significância. Esta abordagem permitiria validar com maior rigor as diferenças observadas entre os algoritmos avaliados.

Por fim, o sistema poderá ser expandido para aplicações mais próximas de cenários reais, incluindo comunicação entre robôs, diferentes modelos cinemáticos, obstáculos dinâmicos mais complexos e integração com plataformas físicas ou simuladores robóticos especializados. Estas extensões permitiriam avaliar o comportamento da arquitetura proposta em ambientes mais próximos das condições encontradas em aplicações industriais e de logística autónoma.

8 Bibliography

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, Chichester: John Wiley & Sons, 2009.
- [2] N. R. Jennings, "An agent-based approach for building complex software systems," *Communications of the ACM*, vol. 44, n° 4, pp. 35-41, 2001.
- [3] J. Gu, M. Goetschalckx e L. F. McGinnis, "Research on warehouse operation: A comprehensive review," *European Journal of Operational Research*, vol. 177, n° 1, pp. 1-21, 2007.
- [4] P. R. Wurman, R. D'Andrea e M. Mountz, "Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses," *AI Magazine*, vol. 29, n° 1, pp. 9-20, 2008.
- [5] S. M. LaValle, Steven M., Cambridge: Cambridge University Press, 2006.
- [6] R. Siegwart, I. R. Nourbakhsh e D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, Cambridge, MA: MIT Press, 2011.
- [7] P. E. Hart, N. J. Nilsson e B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, n° 2, pp. 100-107, 1968.
- [8] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski e R. Barták, "Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks," em *Proceedings of the International Symposium on Combinatorial Search*, Napa, California, 2019.
- [9] G. Sharon, R. Stern, A. Felner e N. R. Sturtevant, "Conflict-Based Search for Optimal Multi-Agent Path Finding," *Artificial Intelligence*, vol. 219, pp. 40-66, 2015.
- [10] D. Silver, "Cooperative Pathfinding," em *Proceedings of the First Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE)*, Marina del Rey, California, 2005.
- [11] C. J. C. H. Watkins e P. Dayan, "Q-learning," *Machine Learning*, vol. 8, n° 3-4, pp. 279-292, 1992.
- [12] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, Cambridge, MA: MIT Press, 2018.

